# UNIT 1

# INTRODUCTION

**Introduction to Operating Systems** – Views of Operating system, Computer System organization, Computer System Architecture; **Operating System Structures** – Operating System Services - User Operating System Interface - System Calls – System Programs - Structuring methods; **Processes** - Process Concept, Process Scheduling, Operations on Processes, Inter-process Communication – Shared Memory Systems, Message Passing Systems, **Threads** - Multithread Models.

## 1.1 INTRODUCTION TO OPERATING SYSTEMS

Operating system plays an important role in the overall computer system. A computer system can be divided roughly into four components: the ***hardware,*** the ***operating system,*** the ***application programs,*** and a ***user*** (Figure 1.1).

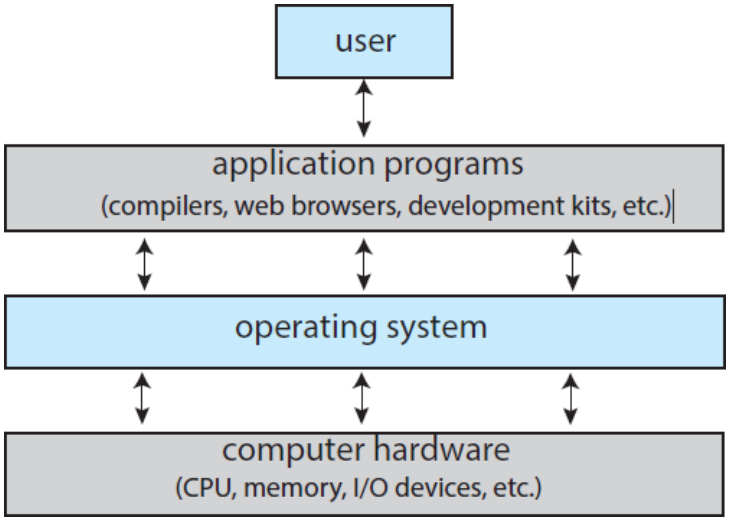


**Figure 1.1 Abstract view of the components of a computer system.**

**Computer hardware**: The central processing unit (CPU), the memory, and the input/output (I/O) devices which provides the basic computing resources for the system.

**Application programs**: Word processors, spreadsheets, compilers, and web browsers are few examples of the application programs which define the ways in which these resources are used to solve users' computing problems.

**Operating system:** Controls the hardware and coordinates its use among the various application programs for the various users. Operating System is a program that acts as an intermediary between a user of a computer and the computer hardware.

**Users:** People, machines or other computers interact with a system through application programs.

Computer system can be viewed as consisting of **hardware, software,** and **data**. The operating system provides the means for proper use of these resources in the operation of the computer system. An operating system is simply provides an ***environment*** within which other programs can do useful work.

### 1.1.1 Views of Operating system

Operating system shall be explored from two viewpoints: User view and System view

- **User View**

The user's view of the computer varies according to the interface being used. Many computer users sit with a laptop or in front of a PC consisting of a monitor, keyboard, and mouse. Such a system is designed for one user to monopolize its resources. The goal is to maximize the work that the user is performing. In this case, the operating system is designed mostly for **ease of use**, with some attention paid to performance and security and none paid to resource utilization—how various hardware and software resources are shared.

Increasingly, many users interact with mobile devices such as smartphones and tablets—devices that are replacing desktop and laptop computer systems for some users. These devices are typically connected to networks through cellular or other wireless technologies. The user interface for mobile computers generally features a **touch screen,** where the user interacts with the system by pressing and swiping fingers across the screen rather than using a physical keyboard and mouse. Many mobile devices also allow users to interact through a **voice recognition interface**, such as Apple's Siri.

Some computers have little or no user view. For example, **embedded computers** in home devices and automobiles may have numeric keypads and may turn indicator lights on or off to show status, but they and their operating systems and applications are designed primarily to run without user intervention.

- **System View**

From the computer's point of view, the operating system is the program most intimately involved with the hardware. In this context, an operating system can be viewed as a **resource allocator**. A computer system has many resources that may be required to solve a problem: CPU time, memory space, storage space, I/O devices, and so on. The operating system acts as the manager of these resources.

Facing numerous and possibly conflicting requests for resources, the operating system must decide how to allocate them to specific programs and users so that it can operate the computer system efficiently and fairly.

A slightly different view of an operating system emphasizes the need to control the various I/O devices and user programs. An operating system is a **control program** manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with the operation and control of I/O devices.

### 1.1.2 Computer System Organization

- A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a **common bus** that provides access to shared memory (Figure 1.2).

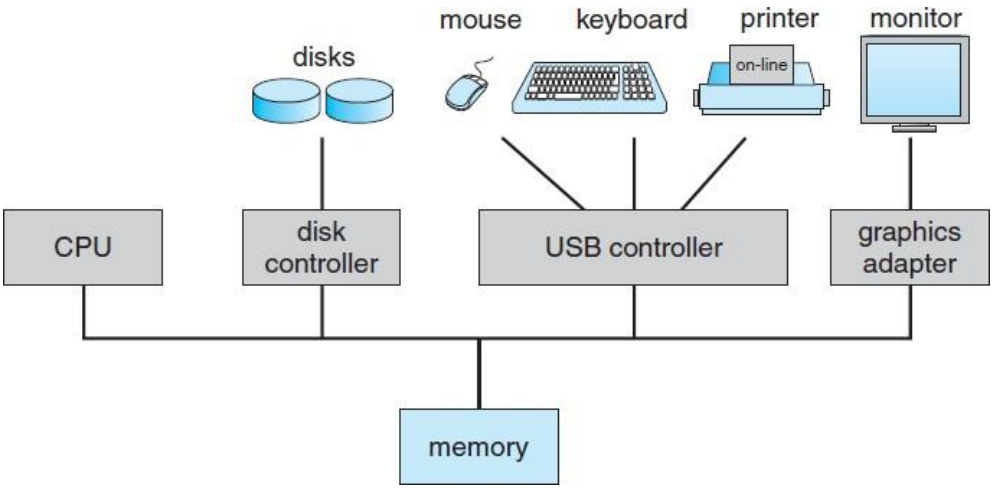


**Figure 1.2 A typical PC computer system.**

- Each **device controller** is in charge of a specific type of device (for example, disk drives, audio devices, or video displays). Depending on the controller, more than one device may be attached. For instance, one system USB port can connect to a USB hub, to which several devices can connect.

- A device controller maintains some **local buffer storage and a set of special-purpose registers**. The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.

- Typically, operating systems have a **device driver** for each device controller. This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device.

- The CPU and the device controllers can execute in parallel, competing for memory cycles. To ensure orderly access to the shared memory, a memory controller synchronizes access to the memory.

Operating a system, focusing on three key aspects named interrupts, storage structure and I/O structure is described below:

### 1.1.2.1 Interrupts

- Consider a typical computer operation: a program performing I/O. To start an I/O operation, the device driver loads the appropriate registers in the device controller. The device controller, in turn, examines the contents of these registers to determine what action to take (such as “read a character from the keyboard”).

- The controller starts the transfer of data from the device to its local buffer. Once the transfer of data is complete, the device controller informs the device driver that it has finished its operation. The device driver then gives control to other parts of the operating system, possibly returning the data or a pointer to the data if the operation was a read. For other operations, the device driver returns status information such as “write completed successfully” or “device busy”.

- For the above scenario, controller inform the device driver about the completion of work, via an **interrupt**.

- Hardware may trigger an interrupt at any time by sending a signal to the CPU, usually by way of the **system bus.**

- Interrupts are used for many other purposes as well and are a key part of how operating systems and hardware interact. When the CPU is interrupted, it stops what it is doing and immediately transfers execution to a fixed location. The fixed location usually contains the starting address where the service routine for the interrupt is located. The interrupt service routine executes; on completion, the CPU resumes the interrupted computation.

- A timeline of this operation is shown in Figure 1.3.

- Interrupts are an important part of a computer architecture. Each computer design has its own interrupt mechanism, but several functions are common. The interrupt must transfer control to the appropriate interrupt service routine. The straightforward method for managing this transfer would be to invoke a generic routine to examine the interrupt information. The routine, in turn, would call the **interrupt-specific handler**.

- However, interrupts must be handled quickly, as they occur very frequently. A table of pointers to interrupt routines can be used instead to provide the necessary speed. The interrupt routine is called indirectly through the table, with no intermediate routine needed.

- Generally, the table of pointers is stored in low memory (the first hundred or so locations). These locations hold the addresses of the interrupt service routines for the various devices. This array, or **interrupt vector**, of addresses is then indexed by a unique number, given with the interrupt request, to provide the address of the interrupt service routine for the interrupting device.

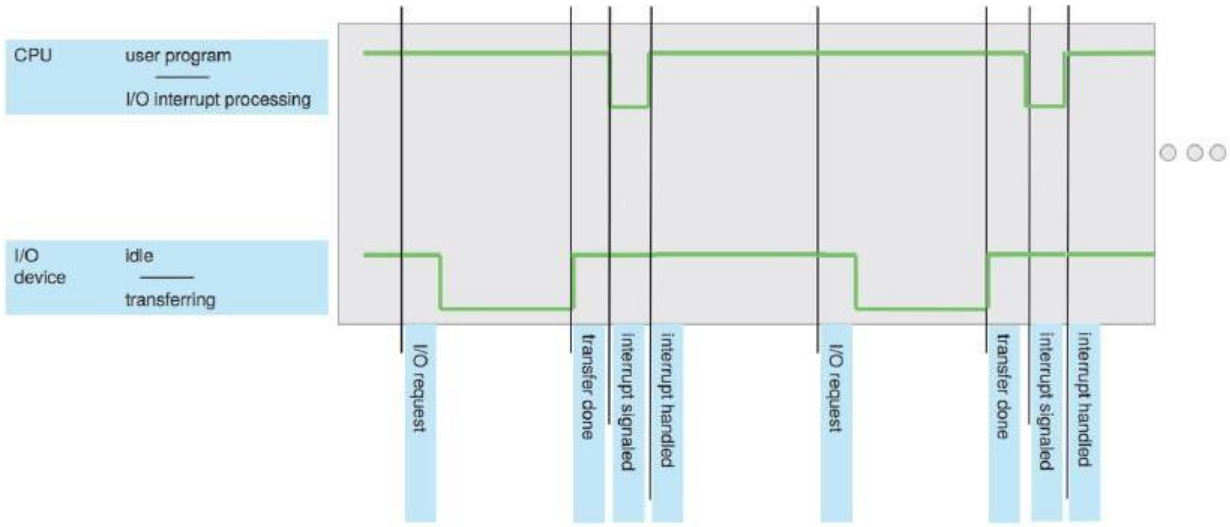


**Figure 1.3 Interrupt timeline for a single program doing output.**

- The interrupt architecture must also save the address of the interrupted instruction. If the interrupt routine needs to modify the processor state—for instance, by modifying register values—it must explicitly save the current state and then restore that state before returning.

- After the interrupt is serviced, the saved return address is loaded into the program counter, and the interrupted computation resumes as though the interrupt had not occurred.

❖ **Implementation of Interrupts**

The basic interrupt mechanism works as follows.

- The CPU hardware has a wire called the **interrupt-request line** that the CPU senses after executing every instruction. When the CPU detects that a controller has asserted a signal on the **interrupt-request line**, it reads the interrupt number and jumps to the **interrupt-handler routine** by using that interrupt number as an index into the interrupt vector.
- It then starts execution at the address associated with that index. The interrupt handler saves any state it will be changing during its operation, determines the cause of the interrupt, performs the necessary processing, performs a state restore, and executes a '***return from interrupt***' instruction to return the CPU to the execution state prior to the interrupt.

- Device controller **raises** an interrupt by asserting a signal on the interrupt request line, the CPU **catches** the interrupt and **dispatches** it to the interrupt handler, and the handler **clears** the interrupt by servicing the device.

- Figure 1.4 summarizes the interrupt-driven I/O cycle.

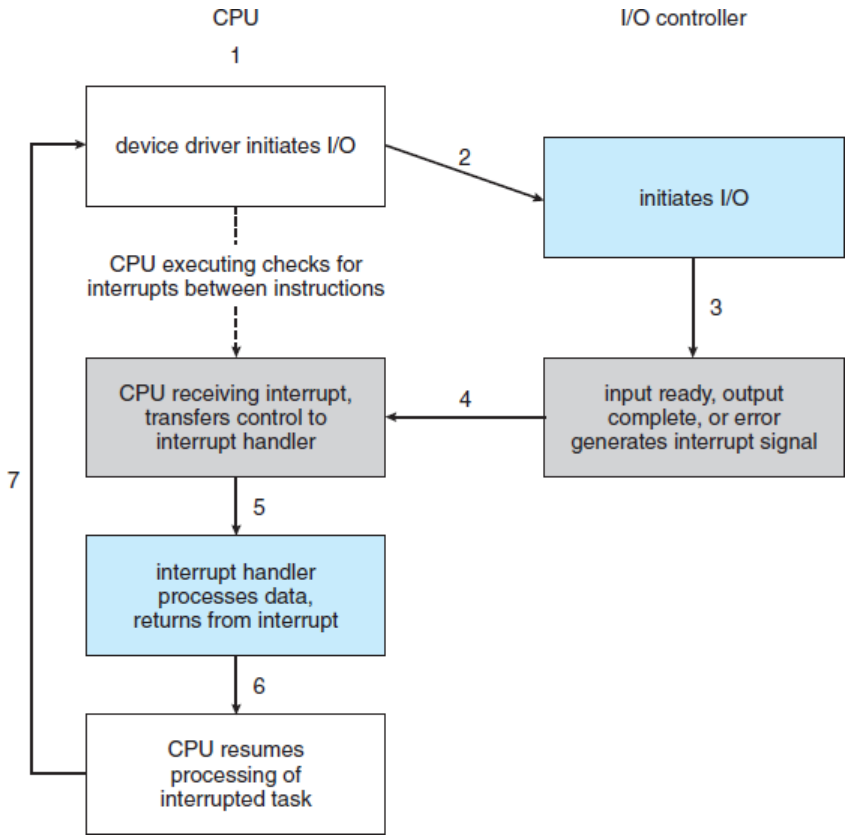


**Figure 1.4 Interrupt-driven I/O cycle.**

- The basic interrupt mechanism just described enables the CPU to respond to an **asynchronous event**, as when a device controller becomes ready for service.

- In a modern operating system, however, more sophisticated interrupt handling features are needed, as listed below.

  1. Ability to defer interrupt handling during critical processing.
  2. Efficient way to dispatch to the proper interrupt handler for a device.
  3. Multilevel interrupts, so that the operating system can distinguish between high- and low-priority interrupts and can respond with the appropriate degree of urgency.

- In modern computer hardware, these three features are provided by the CPU and the **interrupt-controller hardware.**

- Most CPUs have two interrupt request lines. One is the **nonmaskable interrupt**, which is reserved for events such as unrecoverable memory errors.

- The second interrupt line is **maskable:** it can be turned off by the CPU before the execution of critical instruction sequences that must not be interrupted. The maskable interrupt is used by device controllers to request service.

- Purpose of a **vectored interrupt mechanism** is to reduce the need for a single interrupt handler to search all possible sources of interrupts to determine which one needs service. In practice, however, computers have more devices (and, hence, interrupt handlers) than they have address elements in the interrupt vector.

- A common way to solve this problem is to use **interrupt chaining,** in which each element in the interrupt vector points to the head of a list of interrupt handlers. When an interrupt is raised, the handlers on the corresponding list are called one by one, until one is found that can service the request. This structure is a compromise between the overhead of a huge interrupt table and the inefficiency of dispatching to a single interrupt handler.

- The interrupt mechanism also implements a system of **interrupt priority levels**. These levels enable the CPU to defer the handling of low-priority interrupts without masking all interrupts and makes it possible for a high-priority interrupt to preempt the execution of a low-priority interrupt.

### 1.1.2.2 Storage Structure

- The CPU can load instructions only from memory, so any programs must first be loaded into memory to run. General-purpose computers run most of their programs from rewritable memory, called main memory (also called random-access memory, or **RAM).**

- Main memory commonly is implemented in a semiconductor technology called dynamic random-access memory **(DRAM).** Computers use other forms of memory as well.

- For a computer to start running—for instance, when it is powered up or rebooted—it needs to have an initial program, named **bootstrap program** to run. Bootstrap program, loads the operating system.

- The bootstrap program must know how to load the operating system and how to start executing that system. To accomplish this goal, the bootstrap program must locate the operating-system kernel and load it into memory.

- Since RAM is volatile—loses its content when power is turned off or otherwise lost—we cannot trust it to hold the bootstrap program. Typically, it is stored within the computer hardware in read-only memory (**ROM**) or electrically erasable programmable read-only memory (**EEPROM**), known by the general term **firmware**. It initializes all aspects of the system, from CPU registers to device controllers to memory contents.

- All forms of memory provide an **array of bytes.** Each byte has its own address.

- Interaction is achieved through a sequence of **load** or **store** instructions to specific memory addresses. The load instruction moves a byte or word from main memory to an internal register within the CPU, whereas the store instruction moves the content of a register to main memory.

- Aside from explicit loads and stores, the CPU automatically loads instructions from main memory for execution from the location stored in the program counter.

- A typical instruction–execution cycle, as executed on a system with **a von Neumann architecture**, first fetches an instruction from memory and stores that instruction in the **instruction register.** The instruction is then decoded and may cause operands to be fetched from memory and stored in some internal register. After the instruction on the operands has been executed, the result may be stored back in memory.

- Ideally, it is expected programs and data to reside in main memory permanently. This arrangement usually is not possible on most systems for two reasons:

  1. Main memory is usually too small to store all needed programs and data permanently.
  2. Main memory, as mentioned, is volatile—it loses its contents when power is turned off or otherwise lost.

- Thus, most computer systems provide secondary storage as an extension of main memory. The main requirement for secondary storage is that it be able to hold large quantities of data permanently.

- The most common secondary-storage devices are **hard-disk drives** (HDDs) and **nonvolatile memory** (NVM) devices, which provide storage for both programs and data.

- Most programs (system and application) are stored in secondary storage until they are loaded into memory. Many programs then use secondary storage as both the source and the destination of their processing.

- Secondary storage is also much slower than main memory. Hence, the proper management of secondary storage is of central importance to a computer system.

- In a larger sense, however, the storage structure that already described —consisting of registers, main memory, and secondary storage—is only one of many possible storage system designs.

- Other possible components include cache memory, CD-ROM or blu-ray, magnetic tapes, and so on. Those that are slow enough and large enough that they are used only for special purposes —to store backup copies of material stored on other devices, for example—are called **tertiary storage**.

- Each storage system provides the basic functions of storing a datum and holding that datum until it is retrieved at a later time.

- The main differences among the various storage systems lie in **speed, size,** and **volatility.**

- The wide variety of storage systems can be organized in a hierarchy (Figure 1.5) according to storage capacity and access time.

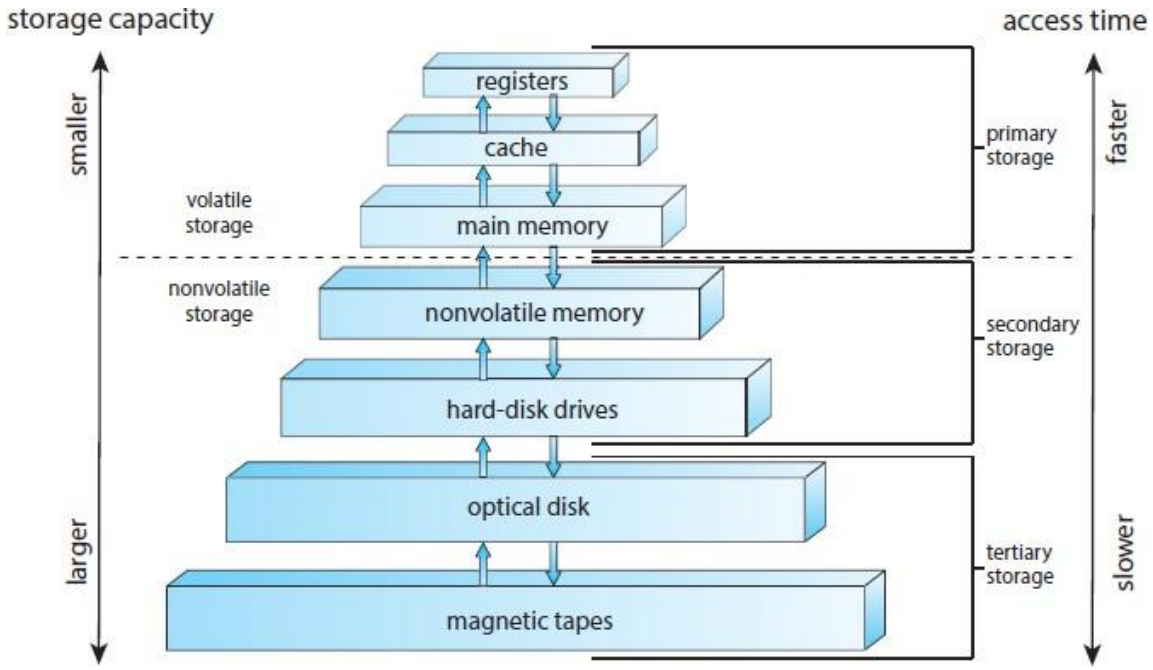


**Figure 1.5 Storage-device hierarchy**

- As a general rule, there is a trade-off between size and speed, with smaller and faster memory closer to the CPU. As shown in the figure, in addition to differing in speed and capacity, the various storage systems are either volatile or nonvolatile. Volatile storage, as mentioned earlier, loses its contents when the power to the device is removed, so data must be written to nonvolatile storage for safekeeping.

- The top four levels of memory in the figure are constructed using **semiconductor** memory, which consists of semiconductor-based electronic circuits.

- NVM devices, at the fourth level, have several variants but in general are faster than hard disks. The most common form of NVM device is flash memory, which is popular in mobile devices such as smartphones and tablets. Increasingly, flash memory is being used for long-term storage on laptops, desktops, and servers as well.

- Important terminologies used in memory management:
  - **Volatile storage** will be referred to simply as memory
  - **Nonvolatile storage** retains its contents when power is lost. This type of secondary storage can be classified into two distinct types:
    - **Mechanical.** A few examples of such storage systems are HDDs, optical disks, holographic storage, and magnetic tape.
    - **Electrical**. A few examples of such storage systems are flash memory, FRAM, NRAM, and SSD.
- Mechanical storage is generally larger and less expensive per byte than electrical storage. Conversely, electrical storage is typically costly, smaller, and faster than mechanical storage.
- The design of a complete storage system must balance all these factors: it must use only as much expensive memory as necessary while providing as much inexpensive, nonvolatile storage as possible.
- Caches can be installed to improve performance where a large disparity in access time or transfer rate exists between two components.

### 1.1.2.3 I/O Structure

- A large portion of operating system code is dedicated to managing I/O, both because of its importance to the reliability and performance of a system and because of the varying nature of the devices.
- A general-purpose computer system consists of multiple devices, all of which exchange data via a common bus. An interrupt-driven I/O is fine for moving small amounts of data but can produce high overhead when used for bulk data movement such as NVS I/O.
- To solve this problem, **direct memory access** (DMA) is used. After setting up buffers, pointers, and counters for the I/O device, the device controller transfers an entire block of data directly to or from the device and main memory, with no intervention by the CPU.
- Only one interrupt is generated per block, to tell the device driver that the operation has completed, rather than the one interrupt per byte generated for low-speed devices. While the device controller is performing these operations, the CPU is available to accomplish other work.

- Some high-end systems use **switch** rather than bus architecture. On these systems, multiple components can talk to other components concurrently, rather than competing for cycles on a shared bus. In this case, DMA is even more effective.

- Figure 1.6 shows the interplay of all components of a computer system.

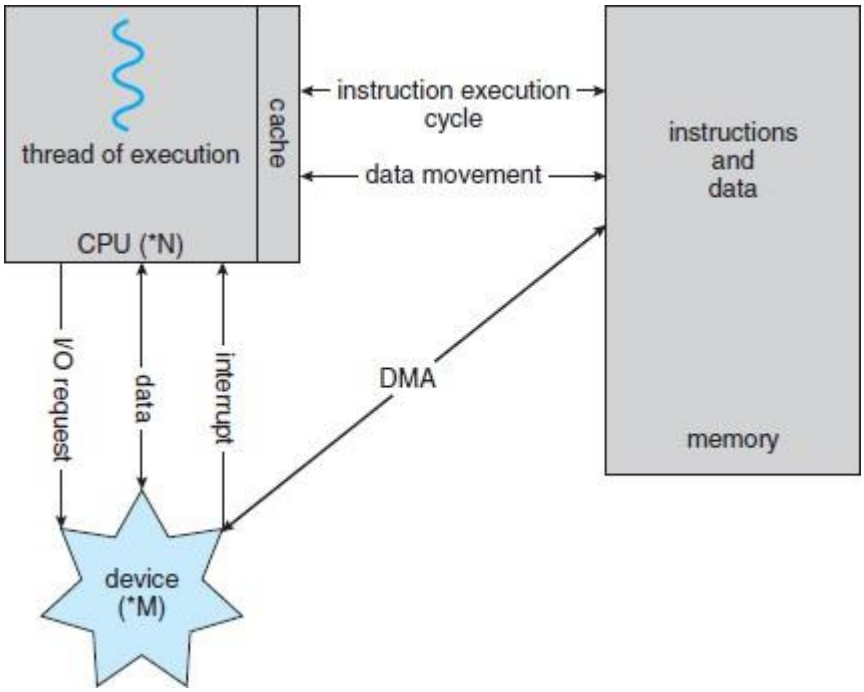


**Figure 1.6 Working of Modern Computer System**

### 1.1.3 Computer-System Architecture

A computer system can be organized in a number of different ways, which can be categorized roughly according to the number of general-purpose processors used.

❖ **Single-Processor Systems**

- Single processor contains one CPU with a single processing core. The core is the component that executes instructions and registers for storing data locally.

- The one main CPU with its core is capable of executing a general-purpose instruction set, including instructions from processes. These systems have other special-purpose process disk, keyboard, and graphics controllers.

- All of these special-purpose processors run a limited instruction set and do not run processes. Sometimes, they are managed by the operating system, in that the operating system sends them information about their next task and monitors their status.

- This arrangement relieves the main CPU of the overhead of disk scheduling. PCs contain a microprocessor in the keyboard to convert the keystrokes into codes to be sent to the CPU.

- In other systems or circumstances, special-purpose processors are low-level components built into the hardware. The operating system cannot communicate with these processors; they do their jobs autonomously. The use of special-purpose microprocessors is common and does not turn a single-processor system into a multiprocessor.

- If there is only one general-purpose CPU with a single processing core, then the system is a single-processor system. According to this definition, however, very few contemporary computer systems are single-processor systems.

❖ **Multiprocessor Systems**

- On modern computers, from mobile devices to servers, multiprocessor systems now dominate the landscape of computing.

- Traditionally, such systems have two (or more) processors, each with a single-core CPU. The processors share the computer bus and sometimes the clock, memory, and peripheral devices.

- The primary advantage of multiprocessor systems is increased throughput. That is, by increasing the number of processors, user expect to get more work done in less time. The speed-up ratio with N processors is not N, however; it is less than N.

- When multiple processors cooperate on a task, a certain amount of overhead is incurred in keeping all the parts working correctly. This overhead, plus contention for shared resources, lowers the expected gain from additional processors.

- The most common multiprocessor systems use **symmetric multiprocessing (SMP),** in which each peer CPU processor performs all tasks, including operating-system functions and user processes.

- Figure 1.7 illustrates a typical SMP architecture with two processors, each with its own CPU. Each CPU processor has its own set of registers, as well as a private—or local— cache.

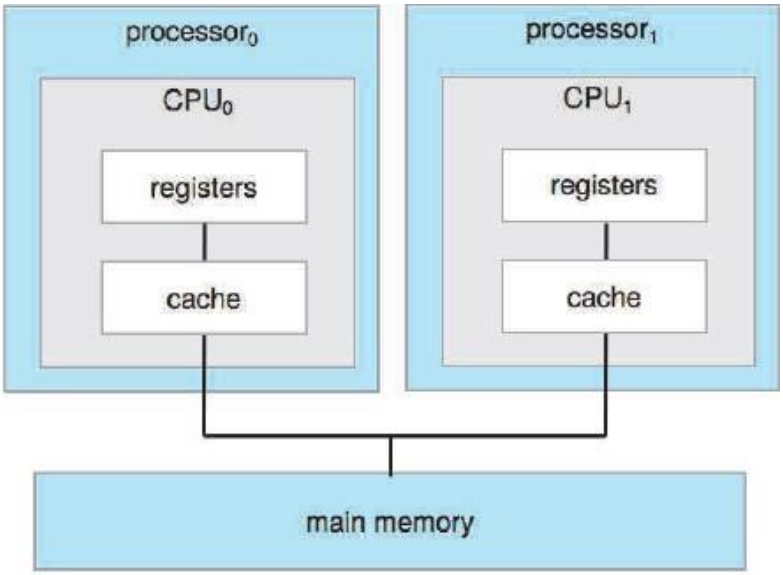


**Figure 1.7 Symmetric multiprocessing architecture**

- However, all processors share physical memory over the system bus. The benefit of this model is that many processes can run simultaneously —N processes can run if there are N CPUs— without causing performance to deteriorate significantly.

- However, since the CPUs are separate, one may be sitting idle while another is overloaded, resulting in inefficiencies. These inefficiencies can be avoided if the processors share certain data structures.

- A multiprocessor system of this form will allow processes and resources—such as memory— to be shared dynamically among the various processors and can lower the workload variance among the processors.

- The definition of multiprocessor has evolved over time and now includes **multicore** systems, in which multiple computing cores reside on a single chip. Multicore systems can be more efficient than multiple chips with single cores because on-chip communication is faster than between-chip communication.

- In addition, one chip with multiple cores uses significantly less power than multiple single-core chips, an important issue for mobile devices as well as laptops.

- Figure 1.8 shows a dual-core design with two cores on the same processor chip. In this design, each core has its own register set, as well as its own local cache, often known as a level 1, or

L1, cache. Notice, too, that a level 2 (L2) cache is local to the chip but is shared by the two processing cores.

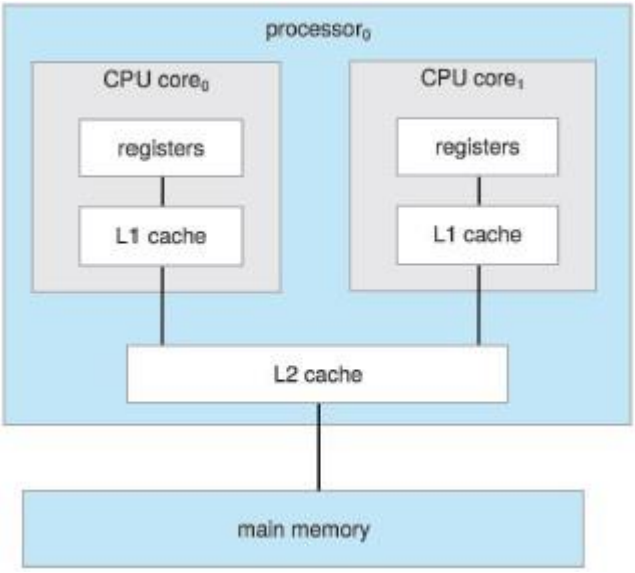


**Figure 1.8 A dual-core design with two cores on the same chip**

- Most architectures adopt this approach, combining local and shared caches, where local, lower-level caches are generally smaller and faster than higher-level shared caches.

- Aside from architectural considerations, such as cache, memory, and bus contention, a multicore processor with N cores appears to the operating system as N standard CPUs.

- Virtually all modern operating systems—including Windows, macOS, and Linux, as well as Android and iOS mobile systems—support multicore SMP systems.

- Adding additional CPUs to a multiprocessor system will increase computing power; however, as suggested earlier, the concept does not scale very well, and once we add too many CPUs, contention for the system bus becomes a bottleneck and performance begins to degrade.

- An alternative approach is instead to provide each CPU (or group of CPUs) with its own local memory that is accessed via a small, fast local bus. The CPUs are connected by a **shared system interconnect**, so that all CPUs share one physical address space. This approach—known as **non-uniform memory access**, or NUMA—is illustrated in Figure 1.9.

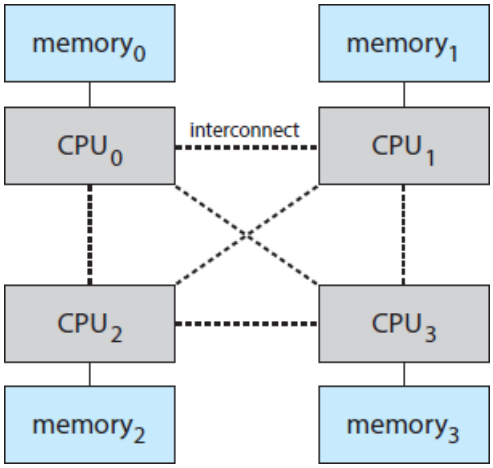


**Figure 1.9 NUMA multiprocessing architecture**

- The advantage is that, when a CPU accesses its local memory, not only is it fast, but there is also no contention over the system interconnect. Thus, NUMA systems can scale more effectively as more processors are added.

- A potential drawback with a NUMA system is increased latency when a CPU must access remote memory across the system interconnect, creating a possible performance penalty. In other words, for example, CPU0 cannot access the local memory of CPU3 as quickly as it can access its own local memory, slowing down performance.

- Operating systems can minimize this NUMA penalty through careful CPU scheduling and memory management. Because NUMA systems can scale to accommodate a large number of processors, they are becoming increasingly popular on servers as well as high-performance computing systems.

- **Blade servers** are systems in which multiple processor boards, I/O boards, and networking boards are placed in the same chassis. The difference between these and traditional multiprocessor systems is that each blade processor board boots independently and runs its own operating system. In essence, these servers consist of multiple independent multiprocessor systems.

### ❖ Clustered Systems

- Another type of multiprocessor system is a clustered system, which gathers together multiple CPUs. Clustered systems differ from the multiprocessor systems that they are composed of two or more individual systems—or nodes—joined together; each node is typically a multicore system. Such systems are considered **loosely coupled**.
-

- The definition of clustered is not concrete; many commercial and open source packages wrestle to define what a clustered system is and why one form is better than another.

- The generally accepted definition is that clustered computers share storage and are closely linked via a local-area network LAN or a faster interconnect, such as InfiniBand.

- Clustering is usually used to provide **high-availability service** that is, service that will continue even if one or more systems in the cluster fail.

- Generally, high availability can be obtained by adding a level of redundancy in the system. A layer of cluster software runs on the cluster nodes. Each node can monitor one or more of the others (over the network).

- If the monitored machine fails, the monitoring machine can take ownership of its storage and restart the applications that were running on the failed machine. The users and clients of the applications see only a brief interruption of service.

- High availability provides increased reliability, which is crucial in many applications. The ability to continue providing service proportional to the level of surviving hardware is called **graceful degradation.**

- Some systems go beyond graceful degradation and are called **fault tolerant**, because they can suffer a failure of any single component and still continue operation. Fault tolerance requires a mechanism to allow the failure to be detected, diagnosed, and, if possible, corrected.

- Clustering can be structured asymmetrically or symmetrically.

- In **asymmetric clustering**, one machine is in **hot-standby mode** while the other is running the applications. The hot-standby host machine does nothing but monitor the active server. If that server fails, the hot-standby host becomes the active server.

- In **symmetric clustering**, two or more hosts are running applications and are monitoring each other. This structure is obviously more efficient, as it uses all of the available hardware. However, it does require that more than one application be available to run.

- Since a cluster consists of several computer systems connected via a network, clusters can also be used to provide **high-performance computing** environments. Such systems can supply significantly greater computational power than single-processor or even SMP systems because they can run an application concurrently on all computers in the cluster.

- The application must have been written specifically to take advantage of the cluster, however. This involves a technique known as **parallelization,** which divides a program into separate components that run in parallel on individual cores in a computer or computers in a cluster.

- Typically, these applications are designed so that once each computing node in the cluster has solved its portion of the problem, the results from all the nodes are combined into a final solution.

- Other forms of clusters include **parallel clusters** and **clustering over a wide-area network** (WAN). Parallel clusters allow multiple hosts to access the same data on shared storage. Because most operating systems lack support for simultaneous data access by multiple hosts, parallel clusters usually require the use of special versions of software and special releases of applications.

- For example, Oracle Real Application Cluster is a version of Oracle's database that has been designed to run on a parallel cluster. Each machine runs Oracle, and a layer of software tracks access to the shared disk. Each machine has full access to all data in the database. To provide this shared access, the system must also supply access control and locking to ensure that no conflicting operations occur. This function, commonly known as a **distributed lock manager** (DLM), is included in some cluster technology.

- Cluster technology is changing rapidly. Some cluster products support thousands of systems in a cluster, as well as clustered nodes that are separated by miles. Many of these improvements are made possible by **storage-area networks** (SANs), which allow many systems to attach to a pool of storage. If the applications and their data are stored on the SAN, then the cluster software can assign the application to run on any host that is attached to the SAN. If the host fails, then any other host can take over. In a database cluster, dozens of hosts can share the same database, greatly increasing performance and reliability.

- Figure 1.10 depicts the general structure of a clustered system.

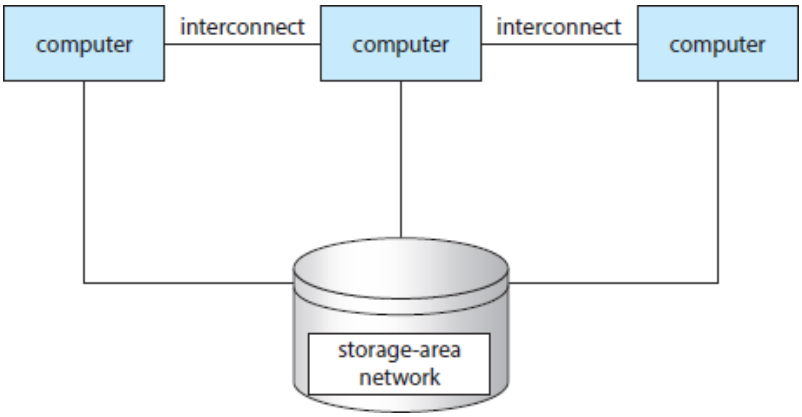


**Figure 1.10 General structure of a clustered system.**

## 1.2 OPERATING SYSTEM STRUCTURES

- An operating system provides the environment within which programs are executed. Internally, operating systems vary greatly in their makeup, since they are organized along many different lines.

- The design of a new operating system is a major task. It is important that the goals of the system be well defined before the design begins. These goals form the basis for choices among various algorithms and strategies.

- Operating system can be viewed from several vantage points.

- One view focuses on the **services** that the system provides; another, on the **interface** that it makes available to users and programmers; a third, on its **components and their interconnections.**

### 1.2.1 Operating-System Services

An operating system provides an environment for the execution of programs. It makes certain services available to programs and to the users of those programs. The specific services provided, of course, differ from one operating system to another, but we can identify common classes.

Figure 1.11 shows one view of the various operating-system services and how they interrelate.

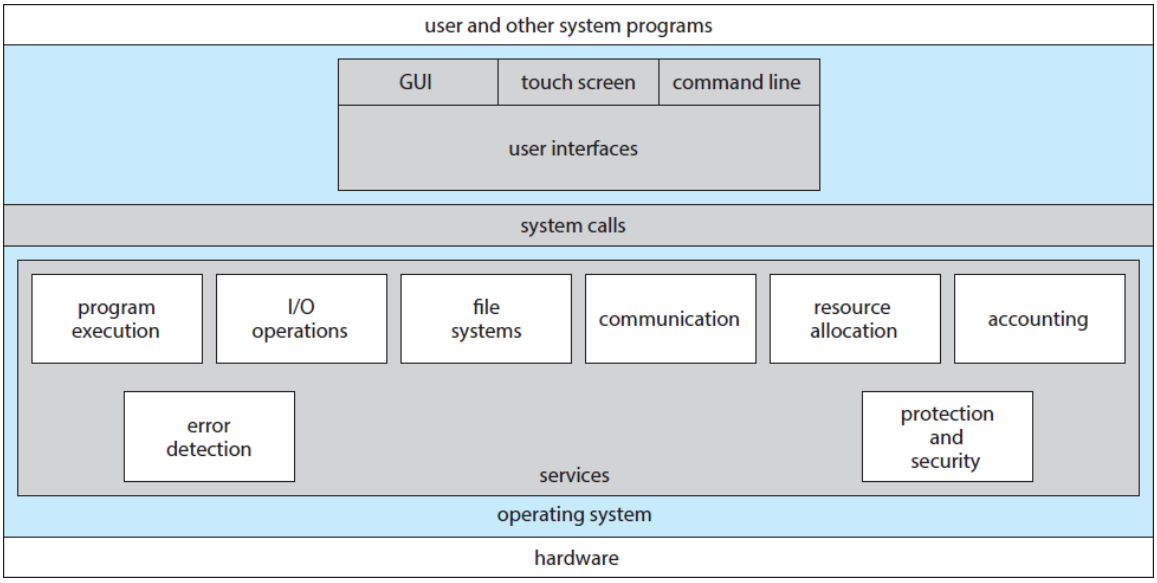


**Figure 1.11 A view of operating system services.**

One set of operating system services provides functions that are **helpful to the user.**

- **User interface.** Almost all operating systems have a user interface (UI). This interface can take several forms.

  Most commonly, a **graphical user interface (GUI)** is used. Here, the interface is a window system with a mouse that serves as a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text.

  Mobile systems such as phones and tablets provide a **touch-screen interface**, enabling users to slide their fingers across the screen or press buttons on the screen to select choices.

  Another option is a **command-line interface (CLI)**, which uses text commands and a method for entering them (say, a keyboard for typing in commands in a specific format with specific options). Some systems provide two or all three of these variations.
- **Program execution.** The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally by indicating an error.
- **I/O operations.** A running program may require I/O, which may involve a file or an I/O device. For specific devices, special functions may be desired (such as reading from a network interface or writing to a file system). For efficiency and protection, users usually cannot control I/O devices directly. Therefore, the operating system must provide a means to do I/O.
- **File-system manipulation.** The file system is of particular interest. Obviously, programs need to read and write files and directories. They also need to create and delete them by name, search for a given file, and list file information.

  Some operating systems include permissions management to allow or deny access to files or directories based on file ownership. Many operating systems provide a variety of file systems, sometimes to allow personal choice and sometimes to provide specific features or performance characteristics.
- **Communications.** There are many circumstances in which one process needs to exchange information with another process. Such communication may occur between processes that are executing on the same computer or between processes that are executing on different computer systems tied together by a network. Communications may be implemented via **shared memory**, in which two or more processes read and write to a shared section of

memory, or **message passing**, in which packets of information in predefined formats are moved between processes by the operating system.

- **Error detection.** The operating system needs to be detecting and correcting errors constantly. Errors may occur in the CPU and memory hardware (such as a memory error or a power failure), in I/O devices (such as a parity error on disk, a connection failure on a network, or lack of paper in the printer), and in the user program (such as an arithmetic overflow or an attempt to access an illegal memory location). For each type of error, the operating system should take the appropriate action to ensure correct and consistent computing.

  Sometimes, it has no choice but to halt the system. At other times, it might terminate an error-causing process or return an error code to a process for the process to detect and possibly correct.

Another set of operating-system functions exists not for helping the user but rather for **ensuring the efficient operation of the system itself**. Systems with multiple processes can gain efficiency by sharing the computer resources among the different processes.

- **Resource allocation.** When there are multiple processes running at the same time, resources must be allocated to each of them. The operating system manages many different types of resources. Some (such as CPU cycles, main memory, and file storage) may have special allocation code, whereas others (such as I/O devices) may have much more general request and release code.
- **Logging.** It is required to keep track of which programs use how much and what kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics. Usage statistics may be a valuable tool for system administrators who wish to reconfigure the system to improve computing services.
- **Protection and security.** The owners of information stored in a multiuser or networked computer system may want to control use of that information. When several separate processes execute concurrently, it should not be possible for one process to interfere with the others or with the operating system itself. Protection involves ensuring that all access to system resources is controlled. Security of the system from outsiders is also important.

Such security starts with requiring each user to authenticate himself or herself to the system, usually by means of a password, to gain access to system resources. It extends to defending external I/O devices, including network adapters, from invalid access attempts and recording all such connections for detection of break-ins. If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.

### 1.2.2 User Operating System Interface

❖ **Command Interpreters**

- Most operating systems, including Linux, UNIX, and Windows, treat the command interpreter as a special program that is running when a process is initiated or when a user first logs on (on interactive systems).

- On systems with multiple command interpreters to choose from, the interpreters are known as **shells.** For example, on UNIX and Linux systems, a user may choose among several different shells, including the C shell, Bourne-Again shell, Korn shell, and others.

- Third-party shells and free user-written shells are also available. Most shells provide similar functionality, and a user's choice of which shell to use is generally based on personal preference.

- The main function of the command interpreter is to get and execute the next user-specified command. Many of the commands given at this level manipulate files: create, delete, list, print, copy, execute, and so on.

- The various shells available on UNIX systems operate in this way. These commands can be implemented in two general ways.

- In one approach, the command interpreter itself contains the code to execute the command. For example, a command to delete a file may cause the command interpreter to jump to a section of its code that sets up the parameters and makes the appropriate system call. In this case, the number of commands that can be given determines the size of the command interpreter, since each command requires its own implementing code.

- An alternative approach—used by UNIX, among other operating systems —implements most commands through **system programs**. In this case, the command interpreter does not understand the command in any way; it merely uses the command to identify a file to be loaded into memory and executed.

- Thus, the UNIX command to delete a file '**rm file.txt**' would search for a file called **rm**, load the file into memory, and execute it with the parameter **file.txt**. The logic associated with the **rm** command would be defined completely by the code in the file rm.

- In this way, programmers can add new commands to the system easily by creating new files with the proper program logic. The command-interpreter program, which can be small, does not have to be changed for new commands to be added.

❖ **Graphical User Interface**

- A second strategy for interfacing with the operating system is through a user friendly graphical user interface, or GUI.

- Here, rather than entering commands directly via a command-line interface, users employ a mouse-based window and menu system characterized by a desktop metaphor.

- The user moves the mouse to position its pointer on images, or icons, on the screen (the desktop) that represent programs, files, directories, and system functions. Depending on the mouse pointer's location, clicking a button on the mouse can invoke a program, select a file or directory—known as a folder—or pull down a menu that contains commands.

- Various GUI interfaces are available for UNIX, however, with significant development in GUI designs from various open-source projects, such as K Desktop Environment (or KDE) and the GNOME desktop by the GNU project.

- Both the KDE and GNOME desktops run on Linux and various UNIX systems and are available under open-source licenses, which means their source code is readily available for reading and for modification under specific license terms.

❖ **Touch-Screen Interface**

- Because a either a command-line interface or a mouse-and-keyboard system is impractical for most mobile systems, smartphones and handheld tablet computers typically use a touch-screen interface.

- Here, users interact by making **gestures** on the touch screen—for example, pressing and swiping fingers across the screen. Although earlier smartphones included a physical keyboard, most smartphones and tablets now simulate a keyboard on the touch screen.

- Both the iPad and the iPhone use the **Springboard** touch-screen interface.

❖ **Choice of Interface**

- The choice of whether to use a command-line or GUI interface is mostly one of personal preference.
- System administrators who manage computers and power users who have deep knowledge of a system frequently use the command-line interface. For them, it is more efficient, giving them faster access to the activities they need to perform.
- Indeed, on some systems, only a subset of system functions is available via the GUI, leaving the less common tasks to those who are command-line knowledgeable.
- Further, command-line interfaces usually make repetitive tasks easier, in part because they have their own programmability. For example, if a frequent task requires a set of command line steps, those steps can be recorded into a file, and that file can be run just like a program. The program is not compiled into executable code but rather is interpreted by the command-line interface. These shell scripts are very common on systems that are command-line oriented, such as UNIX and Linux.
- In contrast, most Windows users are happy to use the Windows GUI environment and almost never use the shell interface. Recent versions of the Windows operating system provide both a standard GUI for desktop and traditional laptops and a touch screen for tablets.
- Historically, Mac OS has not provided a command-line interface, always requiring its users to interface with the operating system using its GUI. However, with the release of macOS (which is in part implemented using a UNIX kernel), the operating system now provides both an Aqua GUI and a command-line interface.
- Although there are apps that provide a command-line interface for iOS and Android mobile systems, they are rarely used.
- Instead, almost all users of mobile systems interact with their devices using the touch-screen interface.
- The user interface can vary from system to system and even from user to user within a system.

### 1.2.3 System Calls

**System calls** provide an interface to the services made available by an operating system. System calls provide the interface between a process and the operating system. When a system call is executed, it is treated as by the hardware as software interrupt. System calls allow user-level processes to request services of the operating system. These calls are generally available as routines written in C and C++.

## ❖ Example of System Calls

Consider an example to illustrate how system calls are used: writing a simple program to read data from one file and copy them to another file. Each of these operations requires a system call.

- The first input that the program will need is the names of the two files: the input file and the output file. These names can be specified in many ways, depending on the operating-system design.

- One approach is to pass the names of the two files as part of the command—for example, the UNIX cp command:

  cp in.txt out.txt

- This command copies the input file in.txt to the output file out.txt.

- A second approach is for the program to ask the user for the names. In an interactive system, this approach will require a sequence of system calls, first to write a prompting message on the screen and then to read from the keyboard the characters that define the two files.

- On mouse-based and icon-based systems, a menu of file names is usually displayed in a window. The user can then use the mouse to select the source name, and a window can be opened for the destination name to be specified. This sequence requires many I/O system calls.
- This system-call sequence is shown in Figure 1.12.

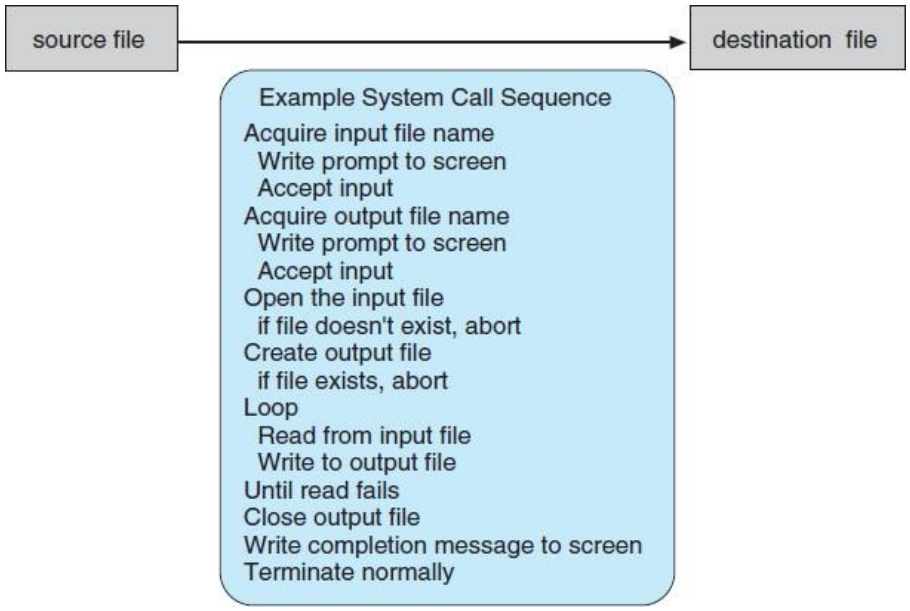


**Figure 1.12 Example of how system calls are used**

- Once the two file names have been obtained, the program must open the input file and create and open the output file. Each of these operations requires another system call.

- Possible error conditions for each system call must be handled. For example, when the program tries to open the input file, it may find that there is no file of that name or that the file is protected against access.

- In these cases, the program should output an error message (another sequence of system calls) and then terminate abnormally (another system call).

- If the input file exists, then a new output file should be created.

- If there is already an output file with the same name, this situation may cause the program to abort (a system call), or we may delete the existing file (another system call) and create a new one (yet another system call).

- Another option, in an interactive system, is to ask the user (via a sequence of system calls to output the prompting message and to read the response from the terminal) whether to replace the existing file or to abort the program.

- When both files are set up, then a loop begins that reads from the input file (a system call) and writes to the output file (another system call). Each read and write must return status information regarding various possible error conditions. On input, the program may find that the end of the file has been reached or that there was a hardware failure in the read (such as a parity error).

- The write operation may encounter various errors, depending on the output device (for example, no more available disk space).

- Finally, after the entire file is copied, the program may close both files (two system calls), write a message to the console or window (more system calls), and finally terminate normally (the final system call).

❖ **Application Programming Interface**

- Even simple programs may make heavy use of the operating system. Frequently, systems execute thousands of system calls per second. Most programmers never see this level of detail.

- Typically, application developers design programs according to an **application programming interface** (API). The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect.

- Three of the most common APIs available to application programmers are the **Windows API** for Windows systems, the **POSIX API** for POSIX-based systems (which include virtually all versions of UNIX, Linux, and macOS), and the **Java API** for programs that run on the Java virtual machine.

- A programmer accesses an API via a library of code provided by the operating system. In the case of UNIX and Linux for programs written in the C language, the library is called libc.

- The functions that make up an API typically invoke the actual system calls on behalf of the application programmer. For example, the Windows function CreateProcess() (used to create a new process) actually invokes the NTCreateProcess() system call in the Windows kernel.

- Another important factor in handling system calls is the **run-time environment** (RTE)—the full suite of software needed to execute applications written in a given programming language, including its compilers or interpreters as well as other software, such as libraries and loaders.

- The RTE provides a **system-call interface** that serves as the link to system calls made available by the operating system. The system-call interface intercepts function calls in the API and invokes the necessary system calls within the operating system.

- Typically, a number is associated with each system call, and the system-call interface maintains a table indexed according to these numbers. The system call interface then invokes the intended system call in the operating-system kernel and returns the status of the system call and any return values.

- The relationship between an API, the system-call interface, and the operating system is shown in Figure 1.13.

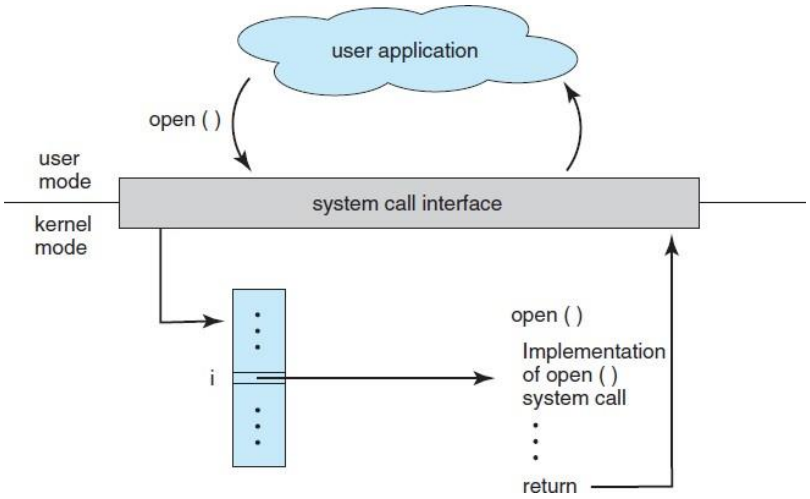


**Figure 1.13 The handling of a user application invoking the open() system call.**

- The caller need know nothing about how the system call is implemented or what it does during execution. Rather, the caller need only obey the API and understand what the operating system will do as a result of the execution of that system call. Thus, most of the details of the operating-system interface are hidden from the programmer by the API and are managed by the run-time support library.

- System calls occur in different ways, depending on the computer in use. Often, more information is required than simply the identity of the desired system call. The exact type and amount of information vary according to the particular operating system and call. For example, to get input, we may need to specify the file or device to use as the source, as well as the address and length of the memory buffer into which the input should be read. Of course, the device or file and length may be implicit in the call.

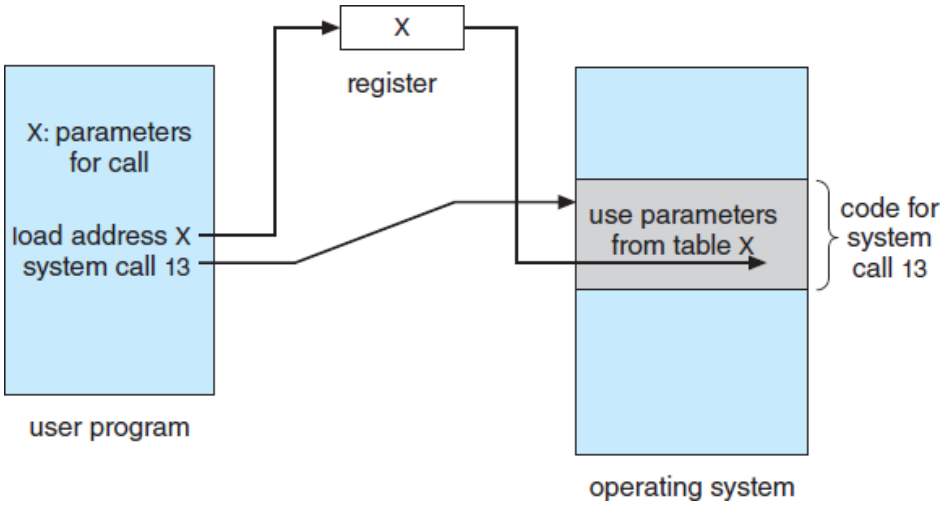


**Figure 1.14 Passing of parameters as a table.**

- Three general methods are used to pass parameters to the operating system. The simplest approach is to pass the parameters in registers. In some cases, however, there may be more parameters than registers. In these cases, the parameters are generally stored in a block, or table, in memory, and the address of the block is passed as a parameter in a register (Figure 1.14).

- Linux uses a combination of these approaches. If there are five or fewer parameters, registers are used. If there are more than five parameters, the block method is used.

- Parameters also can be placed, or pushed, onto a stack by the program and popped off the stack by the operating system. Some operating systems prefer the block or stack method because those approaches do not limit the number or length of parameters being passed.

## ❖ Types of System Calls

System calls can be grouped roughly into six major categories: (i) process control, (ii) file manipulation, (iii) device manipulation, (iv) information maintenance, (v) communications, (vi) protection

The system calls under each group is given below.

• **Process control**

- end, abort

A running program needs to be able to halt its execution either normally **(end())** or abnormally **(abort()).** If a system call is made to terminate the currently running program abnormally, or if the program runs into a problem and causes an error trap, a dump of memory is sometimes taken and an error message generated.

- load, execute

A process executing one program may want to **load() and execute()** another program. This feature allows the command interpreter to execute a program as directed by, for example, a user command or the click of a mouse.

- create process, terminate process

If control returns to the existing program when the new program terminates, it must save the memory image of the existing program; thus, user have effectively created a mechanism for one program to call another program. If both programs continue concurrently, user have created a new process to be multiprogrammed. Often, there is a system call specifically for this purpose **(create process()).** Terminate a process that was already created **(terminate process())** if it is incorrect or is no longer needed.

- get process attributes, set process attributes

If the user create a new process, or perhaps even a set of processes, user should be able to control its execution. This control requires the ability to determine and reset the attributes of a process, including the process's priority, its maximum allowable execution time, and so on using the system calls **get process attributes() and set process attributes().**

- wait for time

Having created new processes, user may need to wait for them to finish their execution. User may want to wait for a certain amount of time to pass using the system call **wait time().**

- wait event, signal event

More probably, user will want to wait for a specific event to occur using the system call **wait event()**. The processes should then signal when that event has occurred using the system call **signal event().**

- Lock and release lock

Quite often, two or more processes may share data. To ensure the integrity of the data being shared, operating systems often provide system calls allowing a process to **lock** shared data. Then, no other process can access the data until the lock is released. Typically, such system calls include acquire **lock() and release lock()**. System calls of these types, dealing with the coordination of concurrent processes.

- Fork,exec,exit

To start a new process, the shell executes a **fork()** system call. Then, the selected program is loaded into memory via an **exec()** system call, and the program is executed. Depending on how the command was issued, the shell then either waits for the process to finish or runs the process "in the background." In the latter case, the shell immediately waits for another command to be entered. When a process is running in the background, it cannot receive input directly from the keyboard, because the shell is using this resource. I/O is therefore done through files or through a GUI interface. Meanwhile, the user is free to ask the shell to run other programs, to monitor the progress of the running process, to change that program's priority, and so on.

When the process is done, it executes an **exit()** system call to terminate, returning to the invoking process a status code of 0 or a nonzero error code.

• **File management**

- create file, delete file

To create() and delete() files, either system call requires the name of the file and perhaps some of the file's attributes.

- open, close

Once the file is created, it is needed to open() it and to use it.

- read, write, reposition

It may also read(), write(), or reposition() (rewind or skip to the end of the file).

- Close

Finally, a system call close() is needed to close the file, indicating that it is no longer to be used.

- get file attributes, set file attributes

For either files or directories, it is required to determine the values of various attributes and perhaps to set them if necessary. File attributes include the file name, file type, protection codes, accounting information, and so on. At least two system calls, get file attributes() and set file attributes(), are required for this function.

- Move, copy

Some operating systems provide many more calls, such as calls for file move() and copy().

• **Device management**

- request device, release device

A process may need several resources to execute—main memory, disk drives, access to files, and so on. If the resources are available, they can be granted, and control can be returned to the user process. Otherwise, the process will have to wait until sufficient resources are available.
A system with multiple users may require us to first request() a device, to ensure exclusive use of it. After finished with the device, release() it.

- read, write, reposition

Once the device has been requested and allocated, then can use read(), write(), and reposition() the device.

- get device attributes, set device attributes

The OS can get such information about a device, say, its status or settings. For example, it can check what is available on disk or the level of the battery of a wireless mouse.

- logically attach or detach devices

This connects a device to the system. It allows the device to be recognized and accessed by processes, e.g., connecting a USB drive to your computer. Logical Detachment: This disconnects a device from the system and makes the device unavailable for use by other processes.

• **Information maintenance**

Many system calls exist simply for the purpose of transferring information between the user program and the operating system. For example, most systems have a system call to return the current **time() and date().**Other system calls may return information about the system, such as the version number of the operating system, the amount of free memory or disk space, and so on.

Another set of system calls is helpful in debugging a program. Many systems provide system calls to **dump()** memory. This provision is useful for debugging. The program strace, which is available on Linux systems, lists each system call as it is executed.

Even microprocessors provide a CPU mode, known as single step, in which a trap is executed by the CPU after every instruction. The trap is usually caught by a debugger.

Many operating systems provide a time profile of a program to indicate the amount of time that the program executes at a particular location or set of locations. A time profile requires either a tracing facility or regular timer interrupts. At every occurrence of the timer interrupt, the value of the program counter is recorded. With sufficiently frequent timer interrupts, a statistical picture of the time spent on various parts of the program can be obtained.

In addition, the operating system keeps information about all its processes, and system calls are used to access this information. Generally, calls are also used to get and set the process information (**get process attributes() and set process attributes()).**

• **Communications**

There are two common models of inter process communication: the **message passing model** and the **shared-memory model**.

In the message-passing model, the communicating processes exchange messages with one another to transfer information. Messages can be exchanged between the processes either directly or indirectly through a common mailbox. Before communication can take place, a connection must be opened. The name of the other communicator must be known, be it another process on the same system or a process on another computer connected by a communications network. Each computer in a network has a host name by which it is commonly known. A host also has a network identifier, such as an IP address. Similarly, each process has a process name, and this name is translated into an identifier by which the operating system can refer to the process. The **get hostid()** and **get processid()** system calls do this translation.

The identifiers are then passed to the general purpose **open()** and **close()** calls provided by the file system or to specific **open connection()** and **close connection()** system calls, depending on the system's model of communication.

The recipient process usually must give its permission for communication to take place with an **accept connection()** call. Most processes that will be receiving connections are special-purpose daemons, which are system programs provided for that purpose. They execute a **wait for connection()** call and are awakened when a connection is made.

The source of the communication, known as the client, and the receiving daemon, known as a server, then exchange messages by using **read message()** and **write message()** system calls. The **close connection()** call terminates the communication.

In the shared-memory model, processes use **shared memory create()** and **shared memory attach()** system calls to create and gain access to regions of memory owned by other processes.

- **Protection**

Protection provides a mechanism for controlling access to the resources provided by a computer system. Historically, protection was a concern only on multiprogrammed computer systems with several users. However, with the advent of networking and the Internet, all computer systems, from servers to mobile handheld devices, must be concerned with protection.

Typically, system calls providing protection include **set permission()** and **get permission(),** which manipulate the permission settings of resources such as files and disks.

The **allow user()** and **deny user()** system calls specify whether particular users can—or cannot—be allowed access to certain resources.

### 1.2.4 System Programs

System programs, also known as system utilities or system services, provide a **convenient environment for program development and execution.** Some of them are simply user interfaces to system calls.

Others are considerably more complex. They can be divided into these categories:

- **File management**. These programs create, delete, copy, rename, print, dump, list, and generally manipulate files and directories.

- **Status information**. Some programs simply ask the system for the date, time, amount of available memory or disk space, number of users, or similar status information. Others are more complex, providing detailed performance, logging, and debugging information. Typically, these programs format and print the output to the terminal or other output devices or files or display it in a window of the GUI. Some systems also support a registry, which is used to store and retrieve configuration information.

- **File modification.** Several text editors may be available to create and modify the content of files stored on disk or other storage devices. There may also be special commands to search contents of files or perform transformations of the text.

- **Programming-language support.** Compilers, assemblers, debuggers, and interpreters for common programming languages (such as C, C++, Java, and PERL) are often provided with the operating system or available as a separate download.

- **Program loading and execution.** Once a program is assembled or compiled, it must be loaded into memory to be executed. The system may provide absolute loaders, relocatable loaders, linkage editors, and overlay loaders. Debugging systems for either higher-level languages or machine language are needed as well.

- **Communications.** These programs provide the mechanism for creating virtual connections among processes, users, and computer systems. They allow users to send messages to one another's screens, to browse Web pages, to send e-mail messages, to log in remotely, or to transfer files from one machine to another.

- **Background services.** All general-purpose systems have methods for launching certain system-program processes at boot time. Some of these processes terminate after completing their tasks, while others continue to run until the system is halted. Constantly running system-program processes are known as services, subsystems, or daemons. Examples include process schedulers that start processes according to a specified schedule, system error monitoring services, and print servers. Typical systems have dozens of daemons.

- In addition, operating systems that run important activities in user context rather than in kernel context may use daemons to run these activities.

- Along with system programs, most operating systems are supplied with programs that are useful in solving common problems or performing common operations.

- Such application programs include Web browsers, word processors and text formatters, spreadsheets, database systems, compilers, plotting and statistical-analysis packages, and games.

- The view of the operating system seen by most users is defined by the application and system programs, rather than by the actual system calls.

### 1.2.5 Operating-System Structure

- A system as large and complex as a modern operating system must be engineered carefully if it is to function properly and be modified easily.
- A common approach is to partition the task into small components, or modules, rather than have one single system.
- Each of these modules should be a well-defined portion of the system, with carefully defined interfaces and functions.

❖ **Monolithic Structure**

The simplest structure for organizing an operating system is no structure at all. That is, place all of the functionality of the kernel into a single, static binary file that runs in a single address space. This approach is known as a **monolithic structure**, is a common technique for designing operating systems.

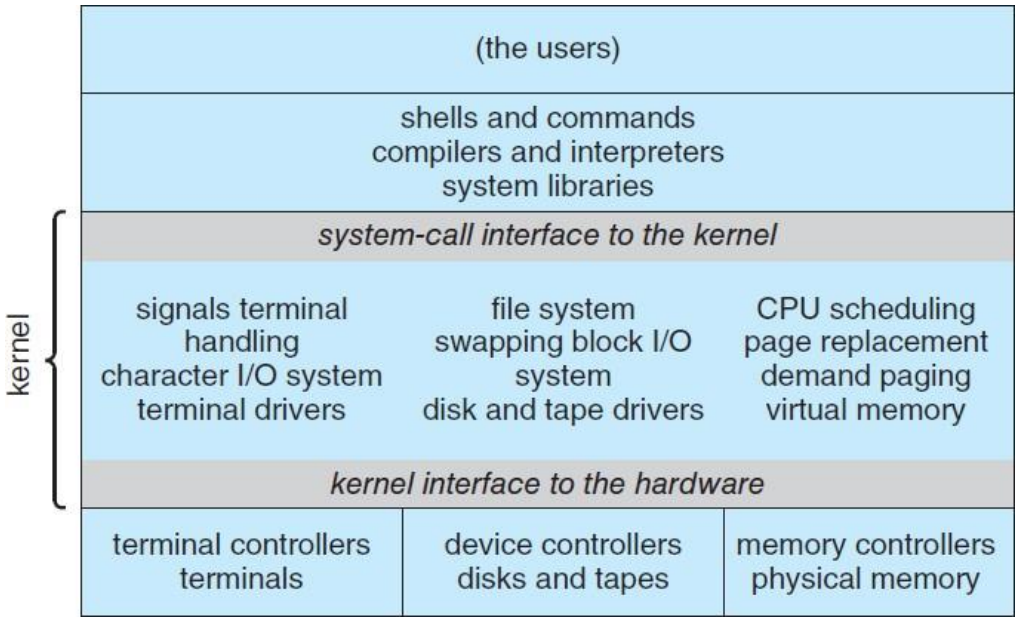


**Figure 1.15 Traditional UNIX system structure**

An example of such limited structuring is the original UNIX operating system, which consists of two separable parts: the kernel and the system programs. The kernel is further separated into a series of interfaces and device drivers, which have been added and expanded over the years as UNIX has evolved.

Traditional UNIX operating system can be viewed as layered to some extent, as shown in Figure 1.15.

Everything below the system-call interface and above the physical hardware is the kernel. The kernel provides the file system, CPU scheduling, memory management, and other operating system functions through system calls. An enormous amount of functionality to be combined into one single address space.

Despite the apparent simplicity of monolithic kernels, they are difficult to implement and extend. Monolithic kernels do have a distinct performance advantage, however: there is very little overhead in the system-call interface, and communication within the kernel is fast.

Therefore, despite the drawbacks of monolithic kernels, their speed and efficiency explains why users see evidence of this structure in the UNIX, Linux, and Windows operating systems.

- **Layered Approach**

The monolithic approach is often known as a **tightly coupled** system because changes to one part of the system can have wide-ranging effects on other parts. Alternatively, we could design a **loosely coupled** system.

Such a system is divided into separate, smaller components that have specific and limited functionality. All these components together comprise the kernel.

The advantage of this modular approach is that changes in one component affect only that component, and no others, allowing system implementers more freedom in creating and changing the inner workings of the system.

A system can be made modular in many ways. One method is the **layered approach**, in which the operating system is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer *N*) is the user interface. This layering structure is depicted in the Figure 1.16.

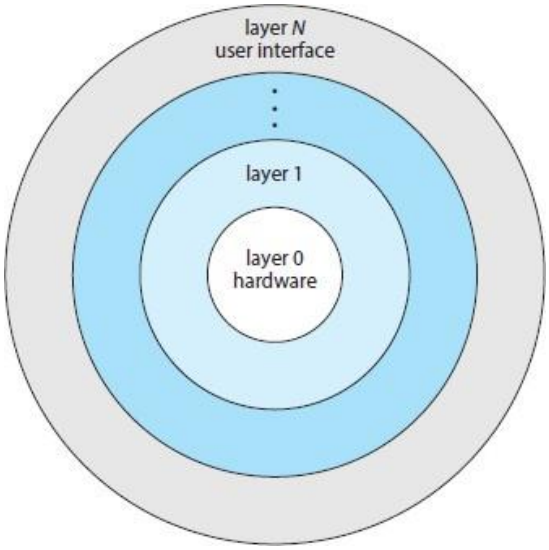


**Figure 1.16 A layered operating system**

An operating-system layer is an implementation of an abstract object made up of data and the operations that can manipulate those data. A typical operating-system layer—say, layer *M*—consists of data structures and a set of functions that can be invoked by higher-level layers. Layer *M*, in turn, can invoke operations on lower-level layers.

The main advantage of the layered approach is simplicity of construction and debugging. The layers are selected so that each uses functions (operations) and services of only lower-level layers.

This approach simplifies debugging and system verification. The first layer can be debugged without any concern for the rest of the system, because, by definition, it uses only the basic hardware (which is assumed correct) to implement its functions. Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on. If an error is found during the debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged. Thus, the design and implementation of the system are simplified.

Each layer is implemented only with operations provided by lower-level layers. A layer does not need to know how these operations are implemented; it needs to know only what these operations do. Hence, each layer hides the existence of certain data structures, operations, and hardware from higher level layers.

- **Microkernels**

As UNIX expanded, the kernel became large and difficult to manage. Operating system called Mach that modularized the kernel using the microkernel approach. This method structures the operating system by removing all nonessential components from the kernel and implementing them as user level programs that reside in separate address spaces. The result is a smaller kernel. There is little consensus regarding which services should remain in the kernel and which should be implemented in user space. Typically, however, microkernels provide minimal process and memory management, in addition to a communication facility. Figure 1.17 illustrates the architecture of a typical microkernel.

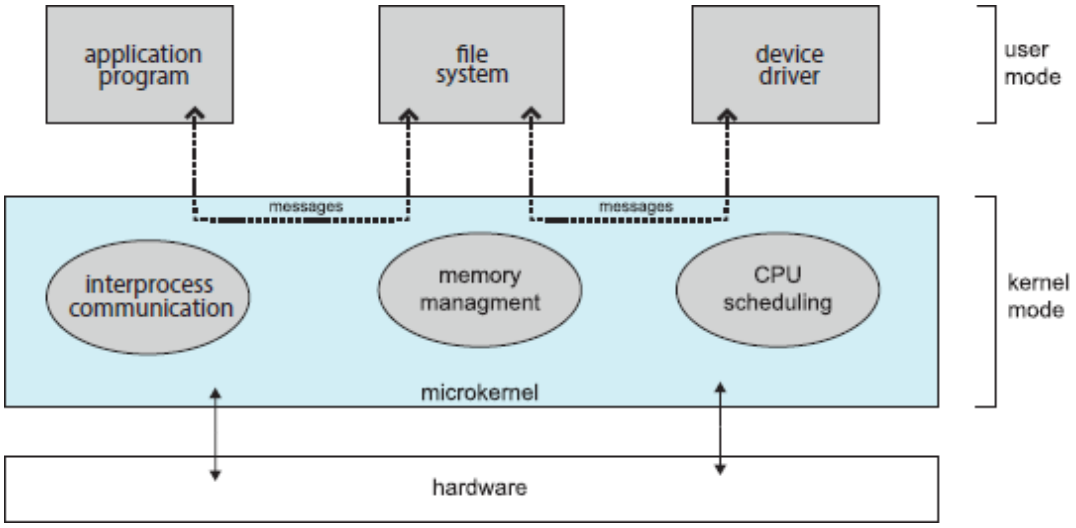


**Figure 1.17 Architecture of a typical microkernel.**

The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space. Communication is provided through message passing.

For example, if the client program wishes to access a file, it must interact with the file server. The client program and service never interact directly. Rather, they communicate indirectly by exchanging messages with the microkernel.

One benefit of the microkernel approach is that it makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel. When the kernel does have to be modified, the changes tend to be fewer, because the microkernel is a smaller kernel. The resulting operating system is easier to port from one hardware design to another. The microkernel also provides more security and reliability, since most services are running as user—rather than kernel—processes. If a service fails, the rest of the operating system remains untouched.

Unfortunately, the performance of microkernels can suffer due to increased system-function overhead. When two user-level services must communicate, messages must be copied between the services, which reside in separate address spaces. In addition, the operating system may have to switch from one process to the next to exchange the messages. The overhead involved in copying messages and switching between processes has been the largest impediment to the growth of microkernel-based operating systems.

- **Modules**

Perhaps the best current methodology for operating-system design involves using loadable kernel modules (LKMs). Here, the kernel has a set of core components and can link in additional services via modules, either at boot time or during run time.

This type of design is common in modern implementations of UNIX, such as Linux, macOS, and Solaris, as well as Windows.

The idea of the design is for the kernel to provide core services, while other services are implemented dynamically, as the kernel is running. Linking services dynamically is preferable to adding newfeatures directly to the kernel, which would require recompiling the kernel every time a change was made.

Thus, for example, we might build CPU scheduling and memory management algorithms directly into the kernel and then add support for different file systems by way of loadable modules.

The overall result resembles a layered system in that each kernel section has defined, protected interfaces; but it is more flexible than a layered system, because any module can call any other module. The approach is also similar to the microkernel approach in that the primary module has only core functions and knowledge of how to load and communicate with other modules; but it is more efficient, because modules do not need to invoke message passing in order to communicate.

- **Hybrid Systems**

Operating systems combine different structures, resulting in hybrid systems that address performance, security, and usability issues. For example, Linux is monolithic, because having the operating system in a single address space provides very efficient performance. However, it also modular, so that new functionality can be dynamically added to the kernel.

Windows is largely monolithic as well (again primarily for performance reasons), but it retains some behavior typical of microkernel systems, including providing support for separate subsystems (known as operating-system personalities) that run as user-mode processes. Windows systems also provide support for dynamically loadable kernel modules.

## 1.3 PROCESSES

- Early computers allowed only one program to be executed at a time. This program had complete control of the system and had access to all the system's resources.
- In contrast, contemporary computer systems allow multiple programs to be loaded into memory and executed concurrently. This evolution required firmer control and more compartmentalization of the various programs; and these needs resulted in the notion of a process, which is a **program in execution**.
- A process is the **unit of work** in a modern computing system.
- The more complex the operating system is, the more it is expected to do on behalf of its users. Although its main concern is the execution of user programs, it also needs to take care of various system tasks that are best done in user space, rather than within the kernel.
- A system therefore consists of a collection of processes, some executing user code, others executing operating system code. Potentially, all these processes can execute concurrently, with the CPU (or CPUs) multiplexed among them.

### 1.3.1 Process Concept

- A process can be thought of as a program in execution.
- A process is the unit of work in a modern time-sharing system.
- A process generally includes the process stack, which contains temporary data (such as method parameters, return addresses, and local variables), and a data section, which contains global variables.

- The memory layout of a process is typically divided into multiple sections, and is shown in Figure 1.18.

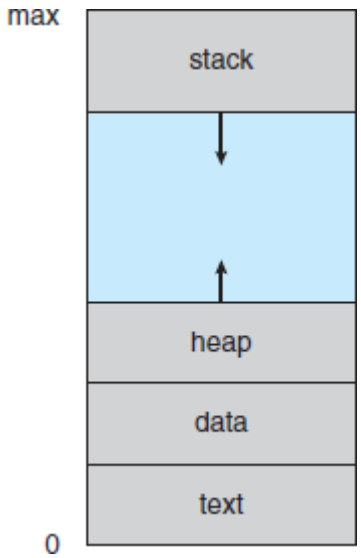


**Figure 1.18 Layout of a process in memory**

- These sections include:

  1. **Text section**—the executable code
  2. **Data section**—global variables
  3. **Heap section**—memory that is dynamically allocated during program run time
  4. **Stack section**—temporary data storage when invoking functions

- Sizes of the text and data sections are fixed, as their sizes do not change during program run time. However, the stack and heap sections can shrink and grow dynamically during program execution.

- Each time a function is called, an activation record containing function parameters, local variables, and the return address is pushed onto the stack; when control is returned from the function, the activation record is popped from the stack. Similarly, the heap will grow as memory is dynamically allocated, and will shrink when memory is returned to the system. Although the stack and heap sections grow toward one another, the operating system must ensure they do not overlap one another.

- A program by itself is not a process. A **program** is a **passive** entity, such as the contents of a file stored on disk, whereas a **process** is an **active** entity, with a program counter specifying the next instruction to execute and a set of associated resources.

- A program becomes a process when an **executable file is loaded into memory**. Two common techniques for loading executable files are double-clicking an icon representing the executable file and entering the name of the executable file on the command line (as in prog.exe or a.out).

- Although two processes may be associated with the same program, they are nevertheless considered two separate execution sequences. For instance, several users may be running different copies of the mail program, or the same user may invoke many copies of the web browser program. Each of these is a separate process; and although the text sections are equivalent, the data, heap, and stack sections vary. It is also common to have a process that spawns many processes as it runs.

❖ **Process States**

- As a process executes, it changes **state**.

- The state of a process is defined in part by the current activity of that process.

- Each process may be in one of the following states: (Figure 1.19)

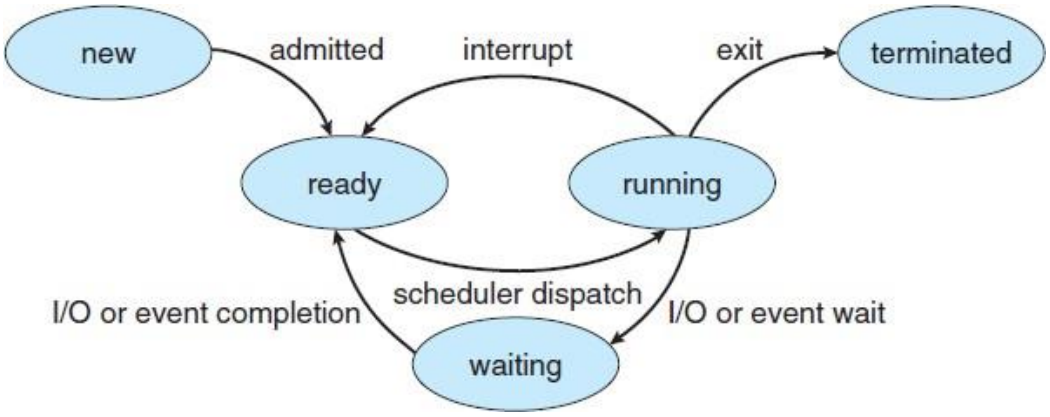


**Figure 1.19 Process State Transition Diagram**

1. **New**: The process is being created.
2. **Running**: Instructions are being executed.
3. **Waiting**: The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
4. **Ready**: The process is waiting to be assigned to a processor.
5. **Terminated**: The process has finished execution.

- It is important to realize that only one process can be **running** on any processor core at any instant.

- Many processes may be **ready** and **waiting**.

## ❖ Process Control Block

- Each process is represented in the operating system by a **process control block** (PCB)- also called a task control block. (Figure 1.20)

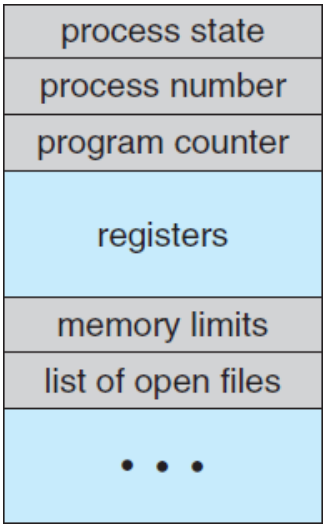


**Figure 1.20 Process control block (PCB)**

- A PCB defines a process to the operating system. It contains the entire information about a process.

- Some of the information a PCB contains are:

1. **Process state**: The state may be new, ready, running, waiting or terminated.
2. **Program counter**: The counter indicates the address of the next instruction to be executed for this process.
3. **CPU registers**: The registers vary in number and type, depending on the computer architecture.
4. **CPU-scheduling information**: This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
5. **Memory-management information**: This information may include such information as the value of the base and limit registers, the page tables, or the segment tables, depending on the memory system used by the operating system.

6. **Accounting information**: This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
7. **I/O Status information**: The information includes the list of I/O devices allocated to this process, a list of open files, and so on.

### 1.3.2 Process Scheduling

- The objective of multiprogramming is to have some process running at all times, so as to **maximize CPU utilization.**

- The objective of time sharing is to switch a CPU core among processes so frequently that users can interact with each program while it is running. To meet these objectives, the **process scheduler** selects an available process (possibly from a set of several available processes) for program execution on a core.

- Each CPU core can run one process at a time.

- For a system with a single CPU core, there will never be more than one process running at a time, whereas a multicore system can run multiple processes at one time.

- If there are more processes than cores, excess processes will have to wait until a core is free and can be rescheduled. The number of processes currently in memory is known as the **degree of multiprogramming**.

- Balancing the objectives of multiprogramming and time sharing also requires taking the general behavior of a process into account. In general, most processes can be described as either **I/O bound** or **CPU bound**.

- An I/O-bound process is one that spends more of its time doing I/O than it spends doing computations. A CPU-bound process, in contrast, generates I/O requests infrequently, using more of its time doing computations.

❖ **<u>Scheduling Queues</u>**

- As processes enter the system, they are put into a **ready queue**, where they are ready and waiting to execute on a CPU's core This queue is generally stored as a linked list; a ready-queue header contains pointers to the first PCB in the list, and each PCB includes a pointer field that points to the next PCB in the ready queue.

- The system also includes other queues. When a process is allocated a CPU core, it executes for a while and eventually terminates, is interrupted, or waits for the occurrence of a particular event, such as the completion of an I/O request. Suppose the process makes an I/O request to a device such as a disk. Since devices run significantly slower than processors, the process will have to wait for the I/O to become available. Processes that are waiting for a certain event to occur — such as completion of I/O — are placed in a **wait queue**

- Ready queue and Wait queue are illustrated in figure 1.21.

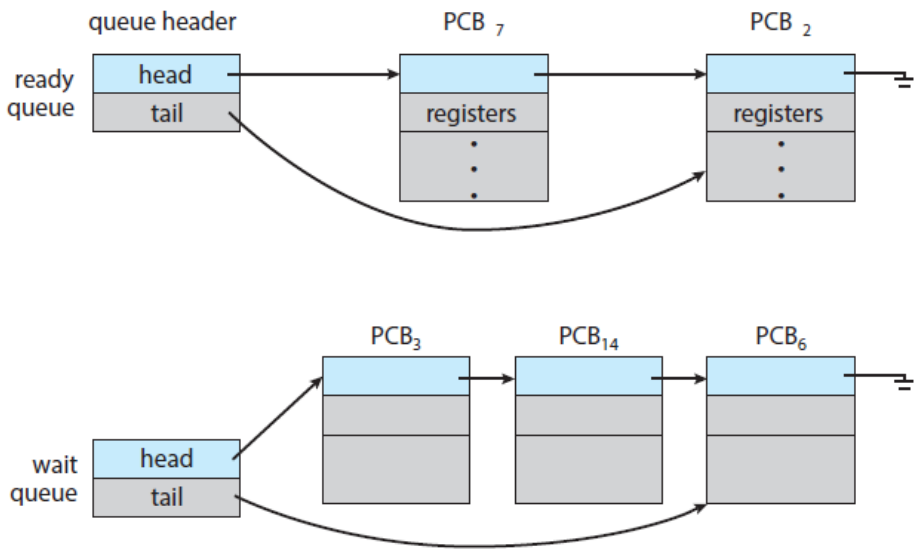


**Figure 1.21 The ready queue and wait queues.**

- A common representation of process scheduling is a **queueing diagram,** shown in Figure 1.22.

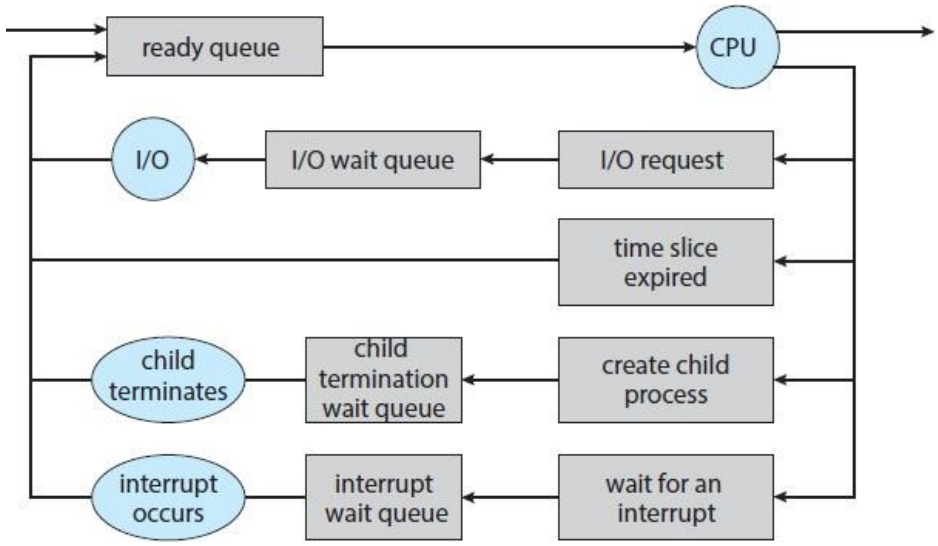


**Figure 1.22 Queueing-diagram representation of process scheduling.**

- Two types of queues are present: the ready queue and a set of wait queues.

- The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.

- A new process is initially put in the ready queue. It waits there until it is selected for execution, or **dispatched**.

- Once the process is assigned to the CPU and is executing, one of several events could occur:
  - The process could issue an I/O request, and then be placed in an I/O queue.
  - The process could create a new subprocess and wait for its termination.
  - The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready Queue.

- In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue.
- A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

## ❖ CPU Scheduling

- A process migrates between the various scheduling queues throughout its lifetime.

- The operating system must select, for scheduling purposes, processes from these queues in some fashion.

- The selection process is carried out by the appropriate **CPU scheduler.**

- There are three different types of schedulers. They are:

  1. Long-term Scheduler or Job Scheduler
  2. Short-term Scheduler or CPU Scheduler
  3. Medium term Scheduler

- The **long-term scheduler**, or **job scheduler**, selects processes from this pool and loads them into memory for execution. It is invoked very infrequently. It controls the degree of multiprogramming.
- The **short-term scheduler**, or **CPU scheduler**, selects from among the processes that are ready to execute, and allocates the CPU to one of them. It is invoked very frequently.
- Some operating systems, such as time-sharing systems, may introduce an additional, intermediate level of scheduling.
- The key idea is **medium-term scheduler**, removes processes from memory and thus reduces the degree of multiprogramming.
- At some later time, the process can be reintroduced into memory and its execution can be continued where it left off. This scheme is called **swapping,** because a process can be "swapped out"

- The CPU scheduler must select a new process for the CPU frequently.

- An I/O-bound process may execute for only a few milliseconds before waiting for an I/O request.

- Although a CPU-bound process will require a CPU core for longer durations, the scheduler is unlikely to grant the core to a process for an extended period.

- Instead, it is likely designed to forcibly remove the CPU from a process and schedule another process to run. Therefore, the CPU scheduler executes at least once every 100 milliseconds, although typically much more frequently.

## ❖ Context Switch

- Interrupts cause the operating system to change a CPU core from its current task and to run a kernel routine. Such operations happen frequently on general-purpose systems.

- When an interrupt occurs, the system needs to save the current context of the process running on the CPU core so that it can restore that context when its processing is done, essentially suspending the process and then resuming it.

- The context is represented in the PCB of the process. It includes the value of the CPU registers, the process state, and memory-management information. Generically, the user perform a state save of the current state of the CPU core, be it in kernel or user mode, and then a state restore to resume operations.

- Switching the CPU to another process requires saving the state of the old process and loading the saved state for the new process. This task is known as a **context switch**, illustrated in figure 1.23.

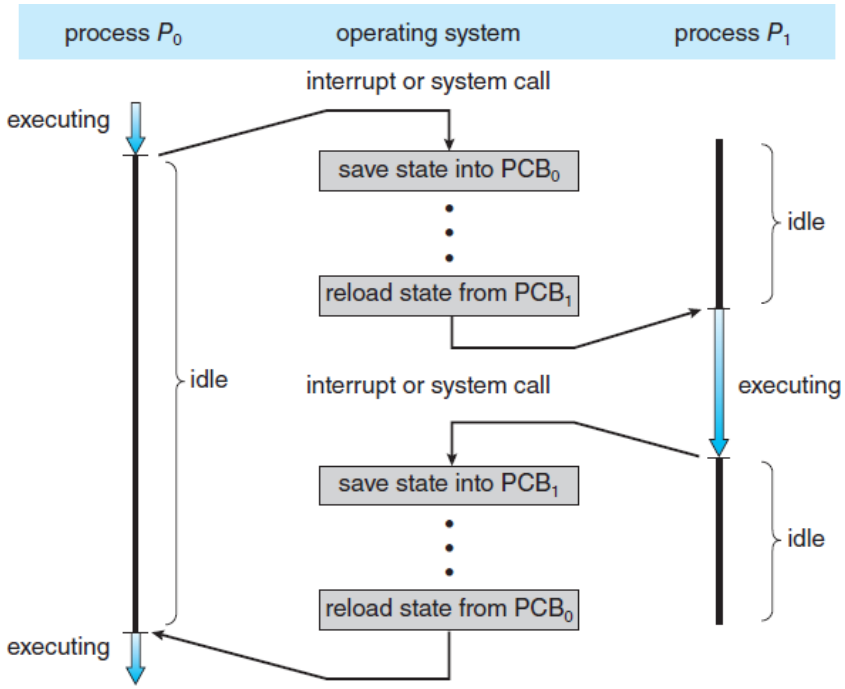


**Figure 1.23 Diagram showing context switch from process to process**

- When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run.

- Context switch time is pure overhead, because the system does no useful work while switching.

- Switching speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions.

- Context-switch times are highly dependent on hardware support. For instance, some processors provide multiple sets of registers. A context switch here simply requires changing the pointer to the current register set.

### 1.3.3 Operations on Processes

- The processes in most systems can execute concurrently, and they may be created and deleted dynamically. Thus, these systems must provide a mechanism for process creation and termination.

❖ **Process Creation**

- A process may create several new processes, during the course of execution.

- The creating process is called a parent process, whereas the new processes are called the children of that process.

- Each of these new processes may in turn create other processes, forming a **tree** of processes.

- Most operating systems (including UNIX, Linux, and Windows) identify processes according to a **unique process identifie**r (or pid), which is typically an integer number.

- The pid provides a unique value for each process in the system, and it can be used as an index to access various attributes of a process within the kernel.

- Figure 1.24 illustrates a typical process tree for the Linux operating system, showing the name of each process and its pid.

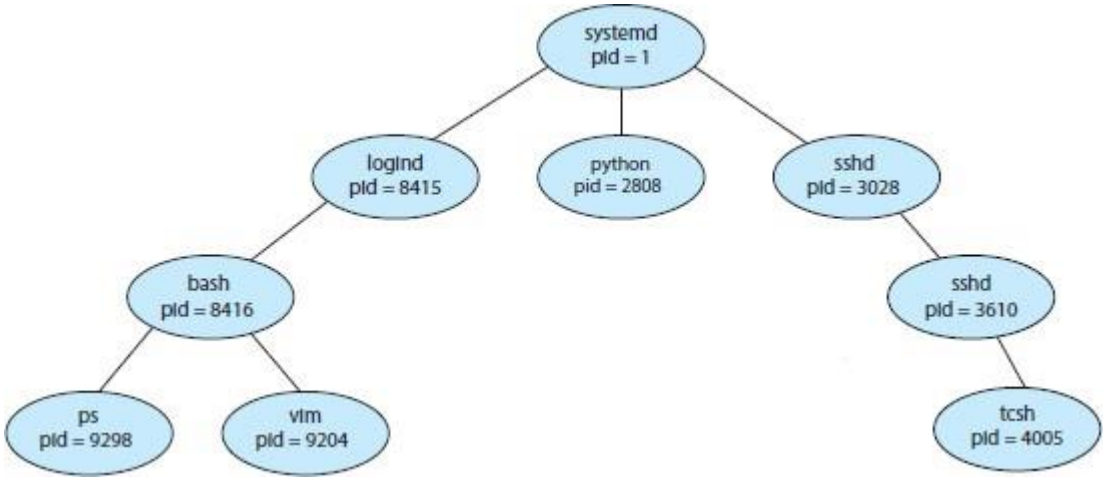


**Figure 1.24 A tree of processes on a typical Linux system.**

- The 'systemd' process (which always has a pid of 1) serves as the root parent process for all user processes, and is the first user process created when the system boots. Once the system has booted, the systemd process creates processes which provide additional services such as a web or print server, an ssh server, and the like.

- In Figure 1.24, there are two children of systemd—logind and sshd. The logind process is responsible for managing clients that directly log onto the system.

- In this example, a client has logged on and is using the bash shell, which has been assigned pid 8416.

- Using the bash command-line interface, this user has created the process ps as well as the vim editor.

- The sshd process is responsible for managing clients that connect to the system by using ssh (which is short for secure shell).

- On UNIX and Linux systems, user can obtain a listing of processes by using the ps command. For example, the command ***ps –el*** will list complete information for all processes currently active in the system.

- In general, when a process creates a child process, that child process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task.

- A child process may be able to obtain its resources directly from the operating system, or it may be constrained to a subset of the resources of the parent process.

- The parent may have to partition its resources among its children, or it may be able to share some resources (such as memory or files) among several of its children.

- Restricting a child process to a subset of the parent's resources prevents any process from overloading the system by creating too many child processes.

- In addition to supplying various physical and logical resources, the parent process may pass along initialization data (input) to the child process.

- When a process creates a new process, two possibilities exist **in terms of execution:**
    1. The parent continues to execute concurrently with its children.
    2. The parent waits until some or all of its children have terminated.

- There are also two possibilities **in terms of the address space** of the new process:
    1. The child process is a duplicate of the parent process.
    2. The child process has a program loaded into it.

- In UNIX, each process is identified by its process identifier, which is a unique integer. A new process is created by the ***fork*** system call.

- The new process consists of a **copy of the address space** of the original process. This mechanism allows the parent process to communicate easily with its child process. Both processes (the parent and the child) continue execution at the instruction after the fork(),with one difference: the return code for the fork() is zero for the new (child) process, whereas the (nonzero) process identifier of the child is returned to the parent.

- After a ***fork()*** system call, one of the two processes typically uses the ***exec()*** system call to **replace the process's memory space** with a new program.

- The ***exec()*** system call loads a binary file into memory (destroying the memory image of the program containing the exec() system call) and starts its execution.

- In this manner, the two processes are able to communicate and then go their separate ways.

- The parent can then create more children; or, if it has nothing else to do while the child runs, it can issue a ***wait()*** system call to move itself off the ready queue until the termination of the child. Because the call to ***exec()*** overlays the process's address space with a new program, ***exec()*** does not return control unless an error occurs.

- Following C code illustrates creating a separate process using the UNIX fork() system call.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
pid t pid;
/* fork a child process */
pid = fork();
if (pid < 0) { /* error occurred */
```

```
fprintf(stderr, "Fork Failed");
return 1;
}
else if (pid == 0) { /* child process */
execlp("/bin/ls","ls",NULL);
}
else { /* parent process */
/* parent will wait for the child to complete */
wait(NULL);
printf("Child Complete");
}
return 0;
}
```

- There are two different processes running copies of the same program. The only difference is that the value of the variable pid for the child process is zero, while that for the parent is an integer value greater than zero (in fact, it is the actual pid of the child process).

- The child process **inherits** privileges and scheduling attributes from the parent, as well certain resources, such as open files. The child process then overlays its address space with the UNIX command /bin/ls (used to get a directory listing) using the ***execlp()*** system call (execlp() is a version of the exec() system call).

- The parent waits for the child process to complete with the wait() system call. When the child process completes (by either implicitly or explicitly invoking exit()), the parent process resumes from the call to wait(), where it completes using the exit() system call. This is illustrated in Figure 1.25.

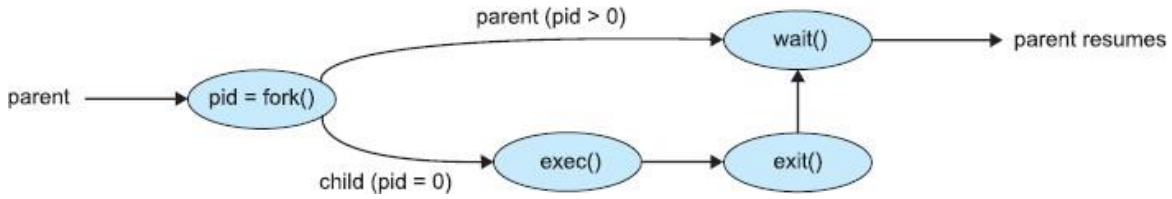


**Figure 1.25 Process creation using the fork() system call.**

- In Window Operating System, processes are created in the **Windows API** using the **CreateProcess()** function, which is similar to fork() in that a parent creates a new child process.

- However, whereas fork() has the child process inheriting the address space of its parent, CreateProcess() requires loading a specified program into the address space of the child process at process creation. Furthermore, whereas fork() is passed no parameters, CreateProcess() expects no fewer than ten parameters.

## ❖ Process Termination

- A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the ***exit()*** system call.

- At that point, the process may return a status value (typically an integer) to its waiting parent process (via the wait system call).

- All the resources of the process —including physical and virtual memory, open files, and I/O buffers—are **deallocated and reclaimed** by the operating system.

- A process can cause the termination of another process via an appropriate system call (for example, **TerminateProcess()** in Windows).

- Usually, such a system call can be invoked only by the parent of the process that is to be terminated. Otherwise, a user—or a misbehaving application—could arbitrarily kill another user's processes.

- Parent needs to know the **identities of its children** if it is to terminate them. Thus, when one process creates a new process, the identity of the newly created process is passed to the parent.

- A parent may terminate the execution of one of its children for a variety of reasons, such as these:
    1. The child has exceeded its usage of some of the resources that it has been allocated.
    2. The task assigned to the child is no longer required.
    3. The parent is exiting, and the operating system does not allow a child to continue if its parent terminates.

- Some systems do not allow a child to exist if its parent has terminated. In such systems, if a process terminates (either normally or abnormally), then all its children must also be terminated. This phenomenon, referred to as **cascading termination**, is normally initiated by the operating system.

- When a process terminates, its resources are deallocated by the operating system. However, its entry in the process table must remain there until the parent calls wait(), because the process table contains the process's exit status. A process that has terminated, but whose parent has not yet called wait(), is known as a **zombie process.**

- All processes transition to this state when they terminate, but generally they exist as zombies only briefly. Once the parent calls wait(), the process identifier of the zombie process and its entry in the process table are released.

- Now consider what would happen if a parent did not invoke wait() and instead terminated, thereby leaving its child processes as **orphans.** Traditional UNIX systems addressed this

scenario by assigning the ***init*** process (which serves as the root of the process hierarchy) as the new parent to orphan processes.

**Process Termination - Android Process Hierarchy**

- Because of resource constraints such as limited memory, mobile operating systems may have to terminate existing processes to reclaim limited system resources.

- Rather than terminating an arbitrary process, Android has identified an importance hierarchy of processes, and when the system must terminate a process to make resources available for a new, or more important, process, it terminates processes in order of increasing importance.

- From most to least important, the hierarchy of process classifications is as follows:

    1. **Foreground process**—The current process visible on the screen, representing the application the user is currently interacting with.
    2. **Visible process**—A process that is not directly visible on the foreground but that is performing an activity that the foreground process is referring to (that is, a process performing an activity whose status is displayed on the foreground process)
    3. **Service process**—A process that is similar to a background process but is performing an activity that is apparent to the user (such as streaming music)
    4. **Background process**—A process that may be performing an activity but is not apparent to the user.
    5. **Empty process**—A process that holds no active components associated with any application

- If system resources must be reclaimed, Android will first terminate empty processes, followed by background processes, and so forth.
- Processes are assigned an importance ranking, and Android attempts to assign a process as high a ranking as possible. For example, if a process is providing a service and is also visible, it will be assigned the more-important visible classification.
- State of a process will be saved prior to termination and resumed at its saved state if the user navigates back to the application.

## 1.4 INTER-PROCESS COMMUNICATION

- The concurrent processes executing in the operating system may be either **independent** processes or **cooperating** processes.

- A process is **independent** if it cannot affect or be affected by the other processes executing in the system. It does not share data with any other processes executing in the system.

- A process is **cooperating** if it can affect or be affected by the other processes executing in the system. This process shares data with other processes.

- **Benefits of Cooperating Processes**

    1. **Information sharing:** Since several applications may be interested in the same piece of information (for instance, copying and pasting), user must provide an environment to allow concurrent access to such information.
    2. **Computation speedup**: If user want a particular task to run faster, user must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing cores.
    3. **Modularity:** User may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.
    4. **Convenience:** Even an individual user may work on many tasks at the same time. For instance, a user may be editing, listening to music, and compiling in parallel.

- Cooperating processes require an **interprocess communication (IPC)** mechanism that will allow them to exchange data— that is, send data to and receive data from each other.

- There are two fundamental models of interprocess communication: **shared memory** and **message passing**.

- In the **shared-memory model,** a region of memory that is shared by the cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.

- In the **message-passing model,** communication takes place by means of messages exchanged between the cooperating processes.

- The two communications models are illustrated in Figure 1.26.

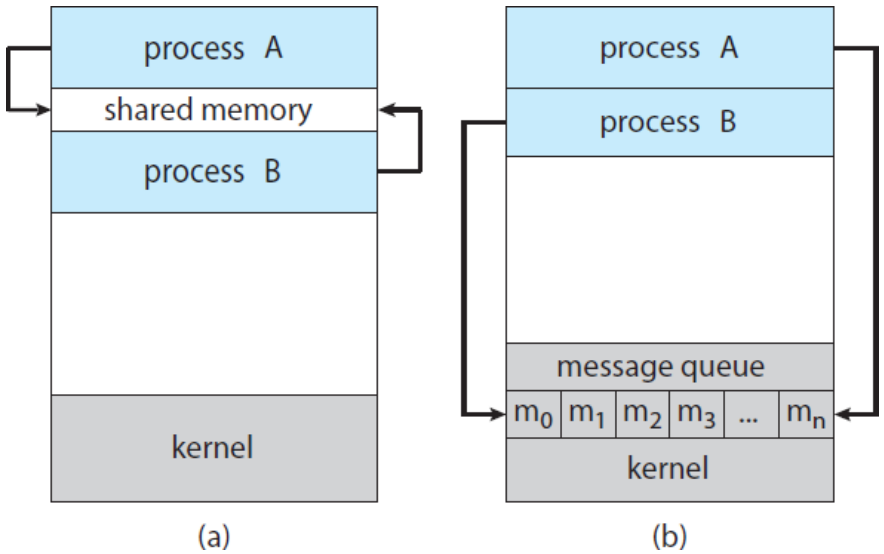


**Figure 1.26 Communications models. (a) Shared memory. (b) Message passing.**

- **Message passing** is useful for exchanging **smaller amounts of data**, because no conflicts need be avoided. Message passing is also easier to implement in a distributed system than shared memory.

- **Shared memory** can be **faster than message passing**, since message-passing systems are typically implemented using system calls and thus require the more time-consuming task of kernel intervention. In shared-memory systems, system calls are required only to establish shared memory regions. Once shared memory is established, all accesses are treated as routine memory accesses, and no assistance from the kernel is required.

### 1.4.1 IPC in Shared-Memory Systems

- Interprocess communication using shared memory requires communicating processes to establish a **region of shared memory**. Typically, a shared-memory region resides in the **address space of the process** creating the shared-memory segment. Other processes that wish to communicate using this shared-memory segment must attach it to their address space.

- Normally, the operating system tries to prevent one process from accessing another process's memory. Shared memory requires that two or more processes agree to remove this restriction. They can then exchange information by reading and writing data in the shared areas. The form of the data and the location are determined by these processes and are not under the operating system's control.

- The processes are also responsible for ensuring that they are not writing to the same location simultaneously.

**Example: Producer – Consumer Problem**

- To illustrate the concept of cooperating processes, let's consider the producer–consumer problem, which is a common paradigm for cooperating processes.

- A producer process produces information that is consumed by a consumer process.

- For example, a print program produces characters that are consumed by the printer driver. A compiler may produce assembly code, which is consumed by an assembler. The assembler, in turn, may produce object modules that are consumed by the loader.
  In client–server paradigm, server is considered as a producer and a client as a consumer.

- One solution to the producer–consumer problem uses shared memory.

- To allow producer and consumer processes to run concurrently, **buffer of items** should be available, that can be **filled** by the producer and **emptied** by the consumer.

- This buffer will reside in a **region of memory** that is shared by the producer and consumer processes. A producer can produce one item while the consumer is consuming another

item.

- The producer and consumer must be **synchronized**, so that the consumer does not try to consume an item that has not yet been produced.

- Two types of buffers can be used.
  - **unbounded-buffer**: places no practical limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items.
  - **bounded-buffer** : assumes that there is a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.

- The following variables reside in a region of memory shared by the producer and consumer processes:

**Shared buffer**

```
#define BUFFER_SIZE 10
typedef struct {
        . . .
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- The shared buffer is implemented as a circular array with two logical pointers: **in** and **out.**

- The variable **in** points to the next free position in the buffer; **out** points to the first full position in the buffer. The buffer is empty when **in** == **out** ; the buffer is full when **((in** + 1) % **BUFFERSIZE)** == **out.**

**Producer Process**

- The code for the producer process is shown below:

```
while (1)
{
  while (((in + 1) % BUFFER_SIZE) == out);
        /* do nothing */
  buffer[in] = nextProduced;
  in = (in + 1) % BUFFER_SIZE;
}
```

- The producer process has a local variable next produced in which the new item to be produced is stored.

**Consumer process**

- The code for the consumer process is shown below:

```
while (1)
{
        while (in == out);
                /* do nothing */
        nextConsumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
}
```

- The consumer process has a local variable next consumed in which the item to be consumed is stored.

### 1.4.2 IPC in Message-Passing Systems

- Operating system to provide the mechanism for cooperating processes to communicate with each other via a message-passing facility.

- Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space. It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.

**Basic Structure:**

- A message-passing facility provides at least two operations: **send(message)** and **receive(message)**

- Messages sent by a process can be either **fixed or variable** in size.

- If only fixed-sized messages can be sent, the system-level implementation is straightforward. This restriction, however, makes the task of programming more difficult. Conversely, variable-sized messages require a more complex system-level implementation, but the programming task becomes simpler. This is a common kind of tradeoff seen throughout operating-system design.

- If processes P and Q want to communicate, they must send messages to and receive messages from each other; a **communication link** must exist between them. Physical implementation of the link is done through a hardware bus , network etc,

- There are several methods for logically implementing a link and the operations:

    1. **Direct or indirect communication**
    2. **Symmetric or asymmetric communication**
    3. **Automatic or explicit buffering**
    4. **Send by copy or send by reference**
    5. **Fixed-sized or variable-sized messages**

❖ **Naming**

- Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication.

- **Direct Communication**

    - Each process that wants to communicate must explicitly name the recipient or sender of the communication.

    - A **communication link** in this scheme has the following properties:

        - A link is established automatically between every pair of processes that want to communicate. The processes need to know only each other's identity to communicate.
        - A link is associated with exactly two processes.
        - Exactly one link exists between each pair of processes.

    - There are two ways of addressing namely **Symmetry in addressing** and **Asymmetry in addressing**

    - **Symmetry in addressing** - that is, both the sender process and the receiver process must name the other to communicate.

    - In symmetry in addressing scheme, the send() and receive() primitives are defined as:
        - send (P, message)—Send a message to process P.
        - receive (Q, message)—Receive a message from process Q.

    - **Asymmetry in addressing** – that is, only the sender names the recipient; the recipient is not required to name the sender.

    - In asymmetry in addressing scheme, the send() and receive() primitives are defined as:
        - send (P, message) - send a message to process P
        - receive(id, message) - receive message from any process, id is set to the name of the process with which communication has taken place

    - The **disadvantage** in both of these schemes (symmetric and asymmetric) is the

limited modularity of the resulting process definitions. Changing the identifier of a process may necessitate examining all other process definitions. All references to the old identifier must be found, so that they can be modified to the new identifier. In general, any such **hard-coding** techniques, where identifiers must be explicitly stated, are less desirable.

- **Indirect Communication**

  - With indirect communication, the messages are sent to and received from **mailboxes**, or **ports.**

  - A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed. Each mailbox has a unique identification.

  - A process can communicate with another process via a number of different mailboxes, but two processes can communicate only if they have a shared mailbox.

  - The send() and receive() primitives are defined as follows:

    - send (A, message)—Send a message to mailbox A.
    - receive (A, message)—Receive a message from mailbox A.

  - A communication link has the following properties:

    - A link is established between a pair of processes only if both members of the pair have a shared mailbox.
    - A link may be associated with more than two processes.
    - Between each pair of communicating processes, a number of different links may exist, with each link corresponding to one mailbox.

  - Now suppose that processes P1, P2, and P3 all share mailbox A. Process P1 sends a message to A, while both P2 and P3 execute a receive() from A. Which process will receive the message sent by P1?

  - The answer depends on which of the following methods chosen:

    - Allow a link to be associated with two processes at most.
    - Allow at most one process at a time to execute a receive() operation.
    - Allow the system to select arbitrarily which process will receive the message (that is, either P2 or P3, but not both, will receive the message). The system may define an algorithm for selecting which process will receive the message (for example, round robin, where processes take turns receiving messages). The system may identify the receiver to the sender.

  - A mailbox may be owned either by a **process** or by the **operating system**. If the mailbox is owned by a process (that is, the mailbox is part of the address space of

the process), then it is possible to distinguish between the owner (which can only receive messages through this mailbox) and the user (which can only send messages to the mailbox).

- Since each mailbox has a unique owner, there can be no confusion about which process should receive a message sent to this mailbox. When a process that owns a mailbox terminates, the mailbox disappears. Any process that subsequently sends a message to this mailbox must be notified that the mailbox no longer exists.

- In contrast, a mailbox that is owned by the operating system has an existence of its own. It is independent and is not attached to any particular process. The operating system then must provide a mechanism that allows a process to do the following:

  - Create a new mailbox.
  - Send and receive messages through the mailbox.
  - Delete a mailbox.

- The process that creates a new mailbox is that mailbox's owner by default. Initially, the owner is the only process that can receive messages through this mailbox. However, the ownership and receiving privilege may be passed to other processes through appropriate system calls. Of course, this provision could result in multiple receivers for each mailbox.

❖ **<u>Synchronization</u>**

- Communication between processes takes place through calls to send() and receive() primitives.

- There are different design options for implementing each primitive. Message passing may be either **blocking or nonblocking** also known as **synchronous and asynchronous.**

  - **Blocking Send** - The sending process blocks itself untill the message is received by the receiving process or by the mail box.
  - **Non-blocking Send** - The sender does not block itself after sending the message but continues with its normal operation. It send the message and resumes operation.
  - **Blocking Receive** - The receiver blocks itself until a message is available
  - **Non-blocking Receive** – The receiver does not block itself. It retrieves either a valid message or a null.

- Code segment of the producer process using message passing is given below:

```
message next produced;
while (true) {
/* produce an item in next produced */
send(next produced);
}
```

- The producer merely invokes the blocking send() call and waits until the message is delivered to either the receiver or the mailbox.

- Code segment of the consumer process using message passing is given below:

```
message next consumed;
while (true) {
receive(next consumed);
/* consume the item in next consumed */
}
```

- When the consumer invokes receive(), it blocks until a message is available.

❖ **<u>Buffering</u>**

- Whether communication is direct or indirect, messages exchanged by communicating processes reside in a **temporary queue**.

- A link has some capacity that determines the number of message that can reside in it temporarily. This property can be viewed as a queue of messages attached to the link.

- Basically, such queues can be implemented in three ways:

  - **Zero capacity** : Queue length of maximum is 0. No message is waiting in a queue. The sender must wait until the recipient receives the message. (message system with no buffering)
  - **Bounded capacity:** The queue has finite length n. Thus at most n messages can reside in it. If the queue is not full when a new message is sent, the message is placed in the queue, and the sender can continue execution without waiting. If the link is full, the sender must block until space is available in the queue.
  - **Unbounded capacity**: The queue has potentially infinite length. Thus any number of messages can wait in it. The sender never blocks.

## 1.5 THREADS

- A thread is a **basic unit of CPU utilization.**

- It is sometimes called as a **lightweight process.**

- It comprises a thread ID, a program counter (PC), a register set, and a stack.

- It **shares with other threads** belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.

- A traditional process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time.

- Most software applications that run on modern computers and mobile devices are multithreaded. An application typically is implemented as a separate process with several threads of control.

- In certain situations, a single application may be required to perform several similar tasks. For example, a web server accepts client requests for web pages, images, sound, and so forth. A busy web server may have several (perhaps thousands of) clients concurrently accessing it.

- If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time, and a client might have to wait a very long time for its request to be serviced.

- One solution is to have the server run as a single process that accepts requests. When the server receives a request, it creates a separate process to service that request. But process creation is time consuming and resource intensive.

- If the web-server process is multithreaded, the server will create a separate thread that listens for client requests. When a request is made, rather than creating another process, the server creates a new thread to service the request and resumes listening for additional requests.

- Figure 1.27 illustrates the difference between a traditional single-threaded process and a multithreaded process.

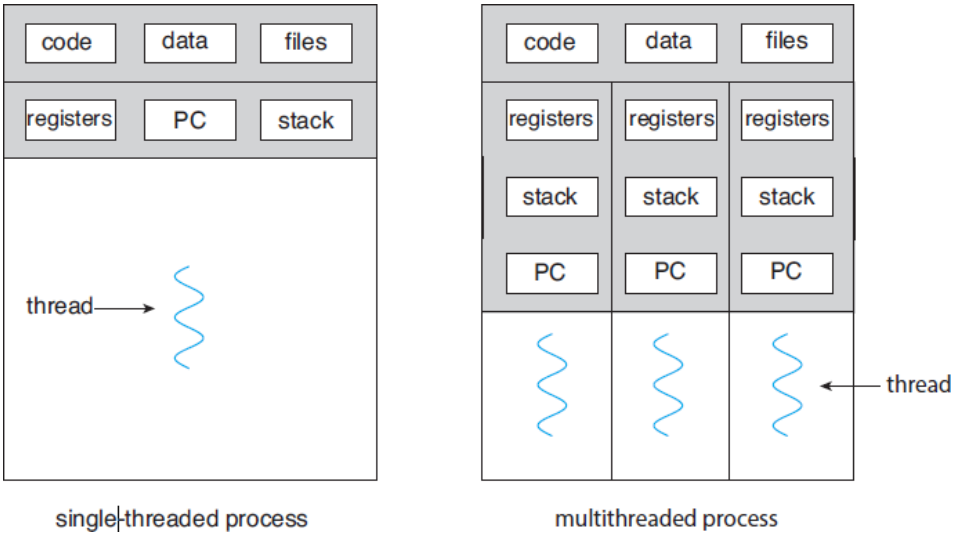


**Figure 1.27 Single-threaded and multithreaded processes**

❖ **Benefits of multithreaded programming**

- **Responsiveness:** Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user. A single-threaded application would be unresponsive to the user until the operation had been completed. In contrast, if the time-consuming operation is performed in a separate, asynchronous thread, the application remains responsive to the user.
- **Resource sharing:** Threads share the memory and the resources of the process to which they belong by default. The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.
- **Economy:** Allocating memory and resources for process creation is costly. Because threads share the resources of the process to which they belong, it is more economical to create and context-switch threads. In general thread creation consumes less time and memory than process creation. Additionally, context switching is typically faster between threads than between processes.
- **Scalability:** The benefits of multithreading can be even greater in a multiprocessor architecture, where threads may be running in parallel on different processing cores. A single-threaded process can run on only one processor, regardless how many are available.

### 1.5.1 Multithreading Models

- Threads may be provided either at the **user level,** for user threads, or by the **kernel level**, for kernel threads.

- User threads are supported above the kernel and are managed without kernel support, whereas kernel threads are supported and managed directly by the operating system.

- Ultimately, a relationship must exist between user threads and kernel threads, as illustrated in Figure 1.28.

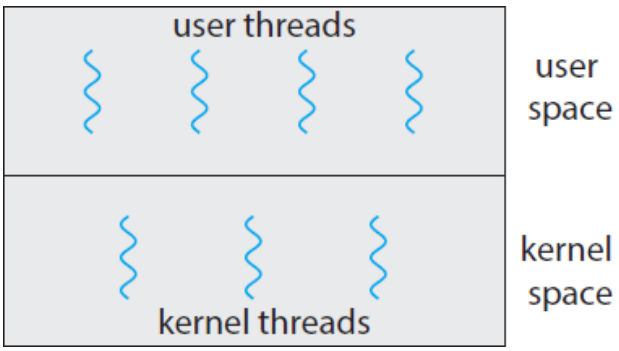


**Figure 1.28 User and kernel threads**

- There are three common ways of establishing such a relationship: the many-to-one model, the one-to one model, and the many-to-many model.

❖ **Many-to-One Model**

- The many-to-one model (Figure 1.29) maps many user-level threads to one kernel thread.

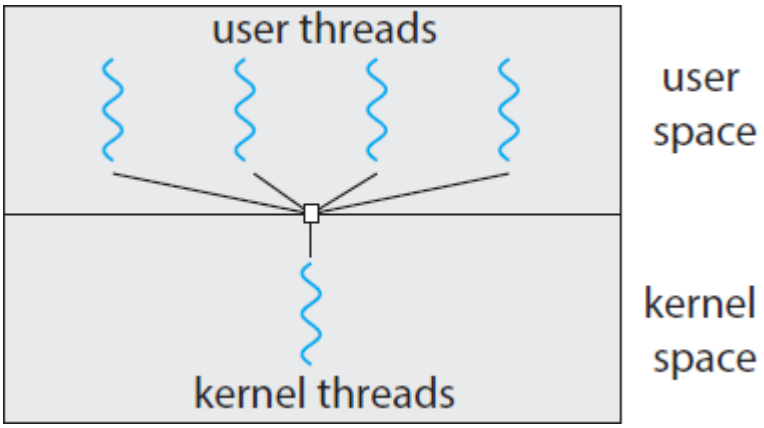


**Figure 1.29 Many-to-one model.**

- Thread management is done by the thread library in user space, so it is efficient.

- However, the entire process will block if a thread makes a blocking system call. Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems.

- **Green threads**—a thread library available for Solaris systems and adopted in early versions of Java—used the many-to one model.

- However, very few systems continue to use the model because of its inability to take advantage of multiple processing cores, which have now become standard on most computer systems.

❖ **One-to-One Model**

- The one-to-one model (Figure 1.30) maps each user thread to a kernel thread. It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call.

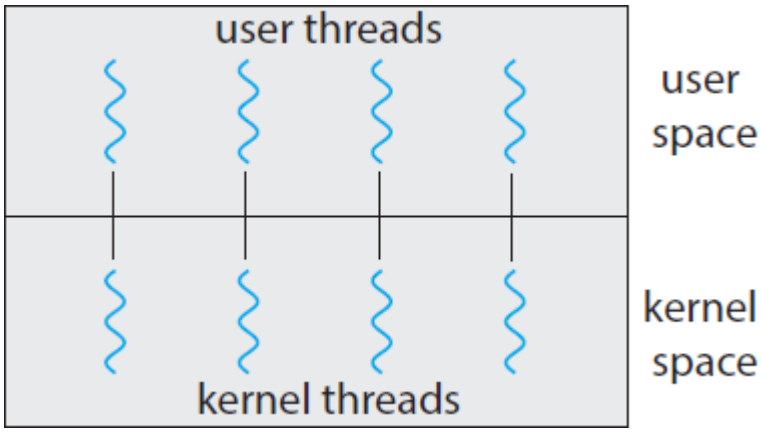


**Figure 1.30 One-to-one model.**

- It also allows multiple threads to run in parallel on multiprocessors.
- The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system.
- Linux, along with the family of Windows operating systems, implement the one-to-one model.

❖ **<u>Many-to-Many Model</u>**

- The many-to-many model (Figure 1.31) multiplexes many user-level threads to a smaller or equal number of kernel threads.
- The number of kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a system with eight processing cores than a system with four cores).

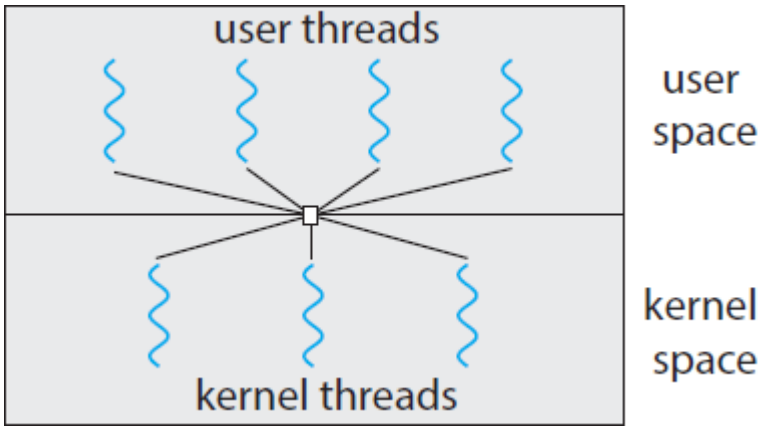


**Figure 1.31 Many-to-many model.**

- Let's consider the effect of this design on concurrency. Whereas the many to- one model allows the developer to create as many user threads as the developer wishes, it does not result in parallelism, because the kernel can schedule only one kernel thread at a time.

- The one-to-one model allows greater concurrency, but the developer has to be careful not to create too many threads within an application. (In fact, on some systems, developer may be limited in the number of threads she can create.)

- The many-to-many model suffers from neither of these shortcomings: developers can create as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor.

- Also, when a thread performs a blocking system call, the kernel can schedule another thread for execution.

- One variation on the many-to-many model still multiplexes many user level threads to a smaller or equal number of kernel threads but also allows a user-level thread to be bound to a kernel thread. This variation is sometimes referred to as the two-level model (Figure 1.32).

- Although the many-to-many model appears to be the most flexible of the models discussed, in practice it is difficult to implement.

- In addition, with an increasing number of processing cores appearing on most systems, limiting the number of kernel threads has become less important.

- As a result, most operating systems now use the one-to-one model.

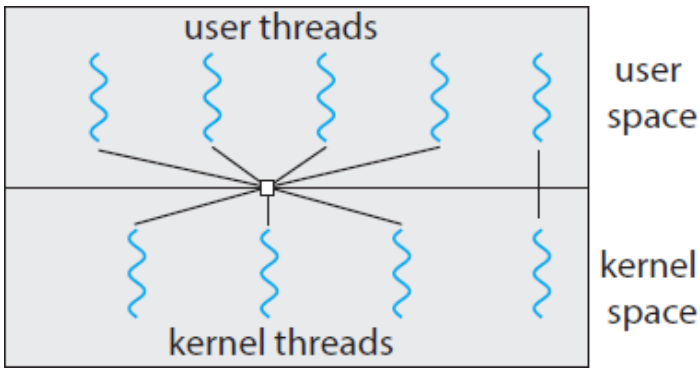


**Figure 1.32 Two-level model**

*********END OF UNIT 1*********